Combustion Reaction Kinetics Computation and Development Documentation Based on OpenFOAM

Name	Affiliation	Email
chizixin	Beihang University	chizixin@buaa.edu.cn

This document was compiled by the author while developing the solver, and is primarily intended for readers interested in combustion reaction kinetics computation, programming, and the optimized implementation of this solver. If you are engaged in numerical simulation related to combustion reaction kinetics, or are interested in the chemical solvers in OpenFOAM, this technical documentation may be worth your time. It covers the following topics:

- **Theoretical aspects**: Primarily introduces the solution of the ODE system under adiabatic constant-pressure combustion.
- **Code-level aspects**: Delves into OpenFOAM implementation details and discusses modifications made to the official code for chemical reaction computations.
- **Performance optimization**: Chapter 2 shares some techniques for vectorization, memory reuse, and mathematical expression reorganization.
- **Verification and testing**: Chapter 3 presents simple result verification and performance testing of the code.

We hope this content is helpful to you. If you have any questions or suggestions, please feel free to reach out.

Combustion Reaction Kinetics Computation Method
- Governing Equations
  - Operator Splitting
  - Ideal Gas
- Instantaneous Rates of Change of State Variables under Adiabatic Constant-Pressure Combustion
  - Choice of Thermodynamic Path
  - Dimension Selection of State Variables
  - Species Mass Fraction
  - Molar Concentration
  - Mole Number
  - OpenFOAM Implementation Details
    - Errors with Molar Concentration
    - Errors with Mass Fraction
- Numerical Solution of Ordinary Differential Equations
  - Explicit Euler Method
  - Implicit Euler Method
  - Linearized Implicit Method
  - Error Estimation and Control
  - OpenFOAM Implementation Details
    - Technical Style
    - Seulex
    - ODE Solver Tolerance Inconsistency
- Jacobian Matrix
  - Species Mass Fraction

- Molar Concentration
- Mole Number
- OpenFOAM Implementation Details
  - Computation of the Jacobian Matrix
  - Effect of the dCdY Approximation on Computational Efficiency
  - Number of Independent Species
- Reaction Kinetics Model
  - Standard Arrhenius Reaction
  - Third-Body-Enhanced Reaction
  - Pressure-Dependent Reaction
  - Unit Conversion
  - OpenFOAM Implementation Details
    - Chemical Equilibrium Constant Limitation
    - JANAF Expression for the Chemical Equilibrium Constant
    - Concentration Product Limitation
    - Effect of Partial Derivatives and Their Approximations on Computational Efficiency
    - Number of Independent Species
- Summary of OpenFOAM Implementation Details

Computational Performance Optimization Strategies

- Mathematical Expression Optimization
  - Rate Constant Evaluation
  - Choice of State Variables
- Code Optimization
  - Instruction-Level Parallelism
  - SIMD
  - Memory Access
  - Thread-Level Parallelism
  - Process-Level Parallelism
- Optimization Examples
  - Forward Rate Constant
  - Concentration Product
  - Chemical Equilibrium Constant
  - Assembling the Partial-Derivative Matrix of Reaction Rates w.r.t. Molar Concentrations and Temperature
  - Thermophysical Property Computation
  - Assembling the Jacobian Matrix
  - LU Decomposition
  - LU Solve
  - Memory Reuse

Tests

- Setup
- Code Validation Tests
- Computational Performance Tests

References

1. Combustion Reaction Kinetics Computation Method

1.1. Governing Equations

For a multi-component chemically reacting system, the species transport equation can be written in the following form

$$\frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{u} Y_i) = \nabla \cdot (\rho D_i \nabla Y_i) + S_i \tag{1}$$

From left to right, the four terms represent, respectively: (i) the rate of change of species mass per unit volume; (ii) the macroscopic advection of the species; (iii) the macroscopic diffusion caused by molecular thermal motion; and (iv) the chemical reactions between molecules. Each symbol is defined as follows:

Symbol	Meaning	Unit
ρ	Density of the mixture	$\mathrm{kg/m^3}$
Y_i	Mass fraction of species i	dimensionless
$\mathbf{u}$	Velocity vector	$\mathrm{m/s}$
D_i	Diffusion coefficient of species i	$\mathrm{m^2/s}$
S_i	Instantaneous chemical reaction source term of species i	$\mathrm{kg/(m^3 \cdot s)}$

The diffusion term is approximated using the Mixture-average model, neglecting thermal diffusion effects and diffusion velocity corrections. In the equation, S_i represents the contribution of chemical reactions to the per-unit-volume mass change rate of species i. It is defined as:

$$S_i = \dot{\omega}_i W_i \tag{2}$$

Here W_i denotes the molar mass of the i-th species, with units of $\mathrm{kg/kmol}$. $\dot{\omega}_i$ denotes the reaction rate obtained from the rate constant and the law of mass action, with units of $\mathrm{kmol/m^3/s}$.

1.1.1. Operator Splitting

Combustion involves a large number of species, including unburned fuel and oxidizer, burned products, and various intermediate species. Among them, the rates of concentration change of certain species are extremely high, far exceeding those induced by advection and diffusion, so conventional discretization schemes cannot be applied directly. Specifically, if, at a given instant, the instantaneous chemical reaction rate of a species is multiplied by the flow time step and the cell volume, the resulting change may be abnormally large or small, exceeding its physically admissible range and leading to negative mass fractions or concentrations; such extreme cases can directly cause the computation to crash. The two simplest approaches to addressing the disparity between the flow and reaction time scales are as follows:

1. Use a sufficiently small flow time step, reaching the same order of magnitude as the smallest chemical reaction time scale, to avoid computational divergence.
2. Adopt the **operator splitting** method: compute the species mass fractions at time $N+1$, then

compute the average chemical reaction rate from time N to time $N+1$ to serve as the chemical reaction source term in the transport equation.

The **operator splitting** method proceeds by first neglecting the advection and diffusion transport terms in the governing equations, retaining only the chemical reaction source term and the unsteady term. This reduces the problem to one governed solely by a system of ordinary differential equations (ODEs). After solving this ODE system, the species mass fractions at the next time instant are obtained, and the time-averaged chemical reaction rate is then computed and used as the chemical reaction source term, as shown below. Note that the expressions below use the density at the initial time to eliminate the effect of volume change. This is because the chemical reaction source term should be the **time-averaged chemical reaction rate**, not the time-averaged **state change rate**; the latter additionally includes the contribution of volume change, as will be explained in detail later.

$$S_i = \rho^N \frac{Y_i^{N+1} - Y_i^N}{\Delta t} \tag{3}$$

Here Y_i^{N+1} denotes the unknown species mass fraction at the next time instant, which is obtained by solving the ODE system. Since chemical reaction rates are typically described by the Arrhenius model and thermal properties require an equation-of-state model, temperature and pressure are also required as state variables. Thus, the following initial value problem must be solved:

$$\begin{cases} \dfrac{d\boldsymbol{\phi}}{dt} = \boldsymbol{f}(\boldsymbol{\phi}), \quad t \in [0, \Delta t] \\ \boldsymbol{\phi}(0) = \boldsymbol{\phi}_0 \end{cases} \tag{4}$$

The state variables in OpenFOAM-9 and earlier versions include species molar concentration ($\mathrm{kmol/m^3}$), temperature (K), and pressure (Pa), expressed as:

$$\boldsymbol{\phi} = (C_1, C_2, ..., C_{Ns}, T, P) \tag{5}$$

The state variables in OpenFOAM-10 and later versions include species mass fraction, temperature, and pressure, expressed as:

$$\boldsymbol{\phi} = (Y_1, Y_2, ..., Y_{Ns}, T, P) \tag{6}$$

Here the subscript Ns denotes the total number of species.

1.1.2. Ideal Gas

The ideal gas equation of state for a multi-component system provides the relationship among density, temperature, and pressure, expressed as:

$$P = C_{tot} R_u T = \left(\sum_{i=1}^{N_s} C_i \right) R_u T \tag{7}$$

Here C_{tot} denotes the total molar concentration ($\mathrm{kmol/m^3}$). Alternatively, in terms of density:

$$P = \rho \frac{R_u}{W_m} T \tag{8}$$

Compared with real gases, from a programming perspective the advantage of the ideal gas lies in the convenience of computing equilibrium constants and mixture thermophysical properties, since the pure-substance thermophysical properties of each species do not interfere with one another. This solver currently focuses only on the ideal gas.

1.2. Instantaneous Rates of Change of State Variables under Adiabatic Constant-Pressure Combustion

1.2.1. Choice of Thermodynamic Path

As mentioned in the previous section, through the operator splitting method we transform the computation of the chemical reaction source term into solving the following form of an ordinary differential equation (ODE) initial value problem:

$$\begin{cases} \dfrac{d\boldsymbol{\phi}}{dt} = \boldsymbol{f}(\boldsymbol{\phi}), \quad t \in [0, \Delta t] \\ \boldsymbol{\phi}(0) = \boldsymbol{\phi}_0 \end{cases} \tag{9}$$

However, **the right-hand side $\boldsymbol{f}(\boldsymbol{\phi})$ of this ODE is not unique**. It reflects the **evolution direction** of the thermodynamic state variables as the chemical reaction proceeds within a closed cell (zero-dimensional homogeneous reactor). Without specifying the concrete physical environment, $\boldsymbol{f}(\boldsymbol{\phi})$ cannot be determined.

Taking the two most common limiting cases as examples:

- **Adiabatic constant-volume combustion**: All heat released by the reaction is used to raise the temperature, while the volume remains fixed, causing a sharp pressure rise. In this case, the change in species concentration is caused solely by the chemical reaction.
- **Adiabatic constant-pressure combustion**: The heat released by the reaction is likewise used to raise the temperature, but the volume is allowed to expand freely to maintain constant pressure. The expansion dilutes the species concentration, thereby affecting the chemical reaction rate in the opposite direction.

In three-dimensional low-Mach-number combustion numerical simulation (such as the OpenFOAM `reactingFoam` solver), the pressure is approximately uniform in space, and the cells are generally treated as having no heat exchange with the surroundings (unless a wall heat-loss model is explicitly added). Therefore, **adiabatic constant-pressure** is the default and most physically realistic closed condition. Under high-Mach-number flows, the physical assumptions must be analyzed on a case-by-case basis; for example, the adiabatic constant-pressure assumption may be used when pressure variations are small in a scramjet engine, while the adiabatic constant-volume assumption may be more appropriate for a detonation engine, and non-equilibrium effects may also need to be considered.

Currently, OpenFOAM-10 determines the unique evolution path based on the **closed adiabatic constant-pressure** condition:

1. Closed condition: No material enters or leaves the cell, so the change in species mass can only result from chemical reactions (which determines the specific form of the $\boldsymbol{f}(\boldsymbol{\phi})$ of species);
2. Adiabatic condition: No energy enters or leaves the cell, and the energy conservation of the combustion process must be satisfied (combined with the constant-pressure condition, the specific expression for the right-hand side of the temperature is determined);
3. Constant-pressure condition: The pressure must remain unchanged during combustion (the specific

expression for the right-hand side of the pressure is determined and is identically zero);

Based on the above constraints, the ODE system can be formulated to determine the system's evolution path; the next step is to solve it. In addition, since the pressure change rate is identically zero, pressure need not be included as a state variable in the ODE system; this solver uses only species and temperature as state variables.

1.2.2. Dimension Selection of State Variables

After determining the adiabatic constant-pressure thermodynamic path, we must also clarify the **dimension** of the state variable ϕ. From the physical constraints, the degrees of freedom of the species in the system are only **Ns-1**, whether expressed in mass fractions or molar concentrations. Therefore, many chemical kinetics solvers select Ns−1 species as independent variables, with the last species passively obtained through the conservation constraint. However, **this project remains consistent with OpenFOAM and adopts all Ns species as independent variables**, i.e., retaining the mathematically redundant dimension. The impact of the two choices on the partial derivative computation will be briefly introduced later.

1.2.3. Species Mass Fraction

Under adiabatic constant-pressure combustion, the instantaneous rate of change of the species mass fraction is:

$$\frac{dY_i}{dt} = \dot{\omega}_i \frac{W_i}{\rho} \tag{10}$$

The instantaneous rate of change of temperature can be derived based on the principle of **constant total enthalpy** under adiabatic constant-pressure combustion, and is expressed as:

$$\frac{dT}{dt} = -\frac{1}{Cp_m} \sum_{i=1}^{i=Ns} Ha_i \frac{dY_i}{dt} \tag{11}$$

Here Cp_m is the mixture constant-pressure specific heat capacity per unit mass ($\mathrm{J/kg/K}$), and Ha_i is the specific absolute enthalpy per unit mass of the i-th species ($\mathrm{J/kg}$).

1.2.4. Molar Concentration

The instantaneous rate of change of the molar concentration ($\mathrm{kmol/m^3/s}$) is given as follows. Here, the second term on the right-hand side represents the contribution from **volume change**, where the volume change itself arises from changes in temperature and concentration. This volume-change term cannot be neglected, because under constant-pressure combustion the total volume expands, causing the rate of change of the molar concentration of inert species to be nonzero.

$$\frac{dC_i}{dt} = \dot{\omega}_i - C_i\left(\frac{1}{C_{tot}} \sum_{j=1}^{j=Ns} \dot{\omega}_j + \frac{1}{T}\frac{dT}{dt}\right) \tag{12}$$

The instantaneous rate of change of temperature can be derived based on the principle of constant total enthalpy, and is expressed as:

$$\frac{dT}{dt} = -\frac{1}{Cp_{vol}} \sum_{i=1}^{i=Ns} ha_i \dot{\omega}_i \tag{13}$$

Here Cp_{vol} is the mixture constant-pressure volumetric heat capacity ($\mathrm{J/m^3/K}$), and ha_i is the specific molar absolute enthalpy of the i-th species ($\mathrm{J/kmol}$).

1.2.5. Mole Number

Although OpenFOAM has never used mole number as a state variable, this project adopts mole number as an alternative scheme (because of its high computational efficiency, as will be explained in detail later). The instantaneous rate of change of mole number (kmol) is:

$$\frac{dN_i}{dt} = \dot{\omega}_i V \tag{14}$$

Here V is the volume of the cell ($\mathrm{m^3}$).

The instantaneous rate of change of temperature can be derived based on the principle of constant total enthalpy, and is expressed as:

$$\frac{dT}{dt} = -\frac{1}{Cp_{tot}} \sum_{i=1}^{i=Ns} ha_i \frac{dN_i}{dt} \tag{15}$$

Here Cp_{tot} is the total constant-pressure heat capacity ($\mathrm{J/K}$).

1.2.6. OpenFOAM Implementation Details

The chemical reaction source term in the species transport equation represents solely the contribution of chemical reactions to changes in species, and should not include contributions from other thermophysical effects. The errors arising from this will be discussed below.

1.2.6.1. Errors with Molar Concentration

OpenFOAM-9 and earlier versions use molar concentration as the state variable. The code block below shows a code snippet from OpenFOAM-7 when using the **instantaneous reaction rate as the chemical reaction source term**:

```
template<class ReactionThermo, class ThermoType>
void Foam::StandardChemistryModel<ReactionThermo, ThermoType>::calculate()
{
    ...
        omega(c_, Ti, pi, dcdt_);

        for (label i=0; i<nSpecie_; i++)
        {
            RR_[i][celli] = dcdt_[i]*specieThermo_[i].W();
        }
    ...
}
```

After analyzing the code, it can be found that this is correct: the chemical reaction source term contains only the contribution of the reaction itself. If the effect of the volume-change term is to be accounted for, it must be reflected in the equation of state and the continuity equation. The rates of change of the state variables mentioned above are defined based on the operator splitting method under **zero-dimensional homogeneous** conditions and are further derived into specific expressions under the closed adiabatic

constant-pressure condition. Under zero-dimensional conditions, the transport terms are eliminated; therefore, volume change can only be represented in source-term form.

The early OpenFOAM versions had problems when computing the **time-averaged reaction rate as the chemical reaction source term**, as detailed below:

1. OpenFOAM-9 and earlier adopted $\frac{dC_i}{dt} = \dot{\omega}_i$ as the ODE expression under ideal-gas adiabatic constant-pressure combustion, which is incorrect; this expression is actually that of **adiabatic constant-volume** combustion, while the temperature equation expression remains that of adiabatic constant-pressure combustion. In other words, the ODE initial value problem solved by the old versions satisfies neither the adiabatic constant-pressure nor the adiabatic constant-volume constraint (even though the code comments state adiabatic constant-pressure combustion); this is thermodynamically inconsistent (refer to the code of the `derivative` and `Jacobian` functions of the chemical reaction model).
2. Using molar concentration as the state variable has a problem when computing the time-averaged chemical reaction rate. The code snippet from OpenFOAM-7 for computing the time-averaged chemical reaction rate as the reaction source term is as follows:

```
template<class ReactionThermo, class ThermoType>
template<class DeltaTType>
Foam::scalar Foam::StandardChemistryModel<ReactionThermo, ThermoType>::solve
(
    const DeltaTType& deltaT
)
{
            //...
            for (label i=0; i<nSpecie_; i++)
            {
                RR_[i][celli] =
                    (c_[i] - c0[i])*specieThermo_[i].W()/deltaT[celli];
            }
            //...
}
```

As can be seen from the above code, even if the correct ODE expression $\frac{dC_i}{dt} = \dot{\omega}_i - C_i(\frac{1}{C_{tot}}\sum_{j=0}^{j=Ns}\dot{\omega}_j + \frac{1}{T}\frac{dT}{dt})$ is used to solve the adiabatic constant-pressure combustion problem, an accurate chemical reaction source term still cannot be obtained, because the change in molar concentration is contributed simultaneously by the chemical reaction and by the volume change caused by thermophysical properties. Numerically, it is impossible to deduce the pure chemical contribution from just two concentration values. In other words, this code computes the time-averaged rate of state change rather than the time-averaged chemical reaction rate. (it can be seen that the molar concentration change rate of inert species is nonzero, leading to a nonzero reaction source term). Therefore, starting with OpenFOAM-10 the state variable was changed to mass fraction to solve this problem, because the mass fraction itself is not governed by volume expansion and contraction.

Finally, note a point that is easily confusing: when we choose to use the **instantaneous chemical reaction rate** as the source term of the species transport equation, using ω_i directly is indeed correct, and the contribution of the volume change term should not be considered in the chemical reaction source term; the volume change is reflected by the continuity equation and the equation of state. However, when using the **time-averaged chemical reaction rate** as the chemical reaction source term of the species transport equation, we need to compute the thermodynamic evolution path of the state variables from the initial state to the next instant based on the adiabatic constant-pressure condition; the resulting evolution path then contains

both the chemical reaction contribution and the volume change contribution. In other words, using molar concentration as the state variable to compute the reaction source term conflicts with the adiabatic constant-pressure combustion condition, and this problem cannot be resolved—we cannot separate out the contribution of volume change. Therefore, OpenFOAM-10 changed the state variable. In the OpenFOAM-10 developer commit record [1], the following description can be seen:

The new form now accounts for the change in reaction rate associated with thermal expansion. Thermal expansion (or contraction) can dilute (or concentrate) the species concentrations, thereby reducing (or increasing) the reaction rates. Previously it was not possible to include this term because it was not computationally feasible to evaluate it in a system in which the state variable was concentration.

1.2.6.2. Errors with Mass Fraction

OpenFOAM-10 incorrectly accounted for the effect of the volume expansion term when computing the average chemical reaction source term:

```
template<class ThermoType>
template<class DeltaTType>
Foam::scalar Foam::chemistryModel<ThermoType>::solve
(
    const DeltaTType& deltaT
)
{
    ...
        for (label i=0; i<nSpecie_; i++)
        {
            RR_[i][celli] = (Y_[i]*rho - Y0[i]*rho0)/deltaT[celli];
        }
    ...
}
```

It can be seen that for inert species, Y_i^{N+1} and Y_i^N are the same (due to mass conservation), but because ρ^{N+1} and ρ^N differ, the chemical reaction source term becomes nonzero.

This problem was resolved in OpenFOAM-11:

```
    for (label i=0; i<nSpecie_; i++)
    {
        RR_[i][celli] = rho0*(Y_[i] - Y0[i])/deltaT[celli];
    }
```

1.3. Numerical Solution of Ordinary Differential Equations

Computing the average chemical reaction rate is essentially solving a system of ordinary differential equations. Numerous methods exist for the numerical solution of ODEs; here we briefly introduce the explicit Euler method, the implicit Euler method, and the linearized implicit Euler method.

1.3.1. Explicit Euler Method

The expression of the explicit Euler method is:

$$\boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N} + \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N}) \tag{16}$$

This method does not require iterative solving and has extremely low computational cost per time step, but its drawback is the extremely strict requirement on the time step size; its stability is limited by the eigenvalue spectrum of the system Jacobian matrix. The stability condition for a linear constant-coefficient system is as follows, where λ_i denotes the i-th eigenvalue (negative).

$$\Delta t \leq \frac{2}{\max\limits_{i} |\mathrm{Re}(\lambda_i)|} \tag{17}$$

Currently OpenFOAM-10 provides the explicit methods Euler, Trapezoid, RKCK45, RKDP45, and RKF45. Combustion reaction computations generally do not use explicit methods, because the time step of explicit methods is limited by the eigenvalues. To illustrate this, consider the following ODE:

$$\frac{d\phi}{dt} = \lambda \phi \tag{18}$$

Here λ is a large negative number (e.g., $\lambda = -10^6$). The analytical solution of this equation is $\phi(t) = \phi_0 e^{\lambda t}$; as t grows, ϕ approaches 0 (the equilibrium point). However, the time step is still limited by the stability condition; even when ϕ^N is as small as 10^{-100} (infinitely close to the equilibrium point), as long as Δt is slightly larger than 2×10^{-6}, the solution will oscillate and its amplitude will keep growing. The implicit and linearized implicit methods do not need to worry about this, because implicit methods can remain stable at large time steps. For the adiabatic constant-pressure combustion problem, the eigenvalues can be obtained by eigen-decomposition of the Jacobian matrix, and the reciprocal of the eigenvalues can serve as the **local characteristic time scale of the linearized system**. For explicit methods, to avoid numerical instability, the time step must be smaller than the minimum chemical reaction time scale of the system, regardless of whether the chemical equilibrium state has been reached.

1.3.2. Implicit Euler Method

The expression of the implicit Euler method is:

$$\boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N} + \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N+1}) \tag{19}$$

For the implicit Euler method, iterative solving is required and can be performed using Newton's iteration method:

$$\begin{cases} \boldsymbol{\phi}_{(k+1)}^{N+1} = \boldsymbol{\phi}_{(k)}^{N+1} - [\mathbf{I} - \Delta t \cdot \mathbf{J}]^{-1} \cdot \boldsymbol{g}(\boldsymbol{\phi}_{(k)}^{N+1}) \\ \boldsymbol{g}(\boldsymbol{\phi}_{(k)}^{N+1}) = \boldsymbol{\phi}_{(k)}^{N+1} - \boldsymbol{\phi}^{N} - \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}_{(k)}^{N+1}) \end{cases} \tag{20}$$

Here the subscript (k) denotes the k-th Newton iteration step, $\mathbf{I}$ denotes the identity matrix, and $\mathbf{J}$ is the Jacobian matrix, representing the partial derivatives of a multivariate function. The Jacobian matrix can be updated at each iteration or computed only once at the beginning; the latter degenerates into the simplified Newton iteration method, whose convergence rate is reduced but which avoids computing the expensive Jacobian matrix—an engineering trade-off is required.

The advantage of the implicit Euler method is that it allows large step sizes, often much larger than explicit time steps in practice. **OpenFOAM-10 does not provide an implicit method that requires iterative solving**; all implicit methods are based on linearization.

1.3.3. Linearized Implicit Method

The linearized implicit Euler method can be obtained by Taylor expansion of the function term of the implicit Euler method:

$$\boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N} + \Delta t \cdot \left[\mathbf{I} - \Delta t \cdot \mathbf{J}^{N}\right]^{-1} \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N}) \tag{21}$$

This method does not require iterative solving while retaining the advantage of the implicit method of allowing large time steps; the linearized implicit method is fairly sensitive to the accuracy of the Jacobian. If an inconsistent or highly erroneous Jacobian is used, the actual order, stability, or efficiency of the method may be degraded. OpenFOAM-10 provides the linearized implicit methods EulerSI, the Rodas series, and the Rosenbrock series. Meanwhile, extrapolation methods such as SIBS and Seulex also use the linearized implicit method internally.

Rearranging the above formula into the following form:

$$\left(\mathbf{I} - \Delta t \mathbf{J}^{N}\right) \boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N} + \Delta t \left[\boldsymbol{f}(\boldsymbol{\phi}^{N}) - \mathbf{J}^{N}\boldsymbol{\phi}^{N}\right] \tag{22}$$

or

$$\left(\mathbf{I}/\Delta t - \mathbf{J}^{N}\right) \boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N}/\Delta t + \left[\boldsymbol{f}(\boldsymbol{\phi}^{N}) - \mathbf{J}^{N}\boldsymbol{\phi}^{N}\right] \tag{23}$$

This formula is the expanded form of solving the linear system **Ax=b**. Since the matrix size is generally small, the LU decomposition method can be used for direct solution. This differs from the conservation equation system, whose discretized matrix is very large and sparse and requires solution methods such as the conjugate gradient method.

1.3.4. Error Estimation and Control

Since the error from numerical discretization increases rapidly with the time step, one cannot simply set $\Delta t = \Delta t_{CFD}$; instead, a smaller ODE sub-time step is used. The ODE sub-time step must be small enough to avoid numerical discretization error, but not so small as to waste efficiency. To this end, the magnitude of the error must be predicted so as to control the step size; the simplest method is to use numerical solvers of the same type but different orders for error estimation. Below is an example using first- and second-order explicit methods; the second-order explicit method is:

$$\boldsymbol{\phi}^{N+1} = \boldsymbol{\phi}^{N} + \frac{\Delta t}{2} \cdot \left[\boldsymbol{f}(\boldsymbol{\phi}^{N}) + \boldsymbol{f}\left(\boldsymbol{\phi}^{N} + \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N})\right)\right] \tag{24}$$

The difference between the predicted values of the second-order and first-order explicit methods can be approximated as an estimate of the **local truncation error** of the first-order explicit method:

$$\boldsymbol{\epsilon}^{N+1} = \frac{\Delta t}{2} \cdot \left[\boldsymbol{f}(\boldsymbol{\phi}^{N}) + \boldsymbol{f}\left(\boldsymbol{\phi}^{N} + \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N})\right)\right] - \Delta t \cdot \boldsymbol{f}(\boldsymbol{\phi}^{N}) \tag{25}$$

After obtaining the estimate of the current local truncation error, OpenFOAM uses the following expression to obtain the normalized local truncation error:

$$\epsilon_{\max} = \max_{i} \left\{ \frac{|\epsilon_i|}{\epsilon_{\mathrm{abs},i} + \epsilon_{\mathrm{rel},i} \cdot \max\left(|\phi_{0,i}|,\ |\phi_i|\right)} \right\} \tag{26}$$

When $\epsilon_{\max}$ is greater than 1, it indicates that the predicted local error is too large at this point; the result of this round must be discarded, the ODE step size reduced, and the computation restarted from the initial state. For state variables of large magnitude, such as temperature, the relative tolerance term $\epsilon_{\mathrm{rel},i} \cdot \max\left(|\phi_{0,i}|,\ |\phi_i|\right)$ in the normalized error expression dominates. For example, if the relative and absolute tolerances are set to 0.1 and 10^{-8}, and the initial temperature is 999 K with a predicted temperature of 1000 K, then the temperature deviation allowed by the relative tolerance is 100 K, while that allowed by the absolute tolerance is 10^{-8} K; according to the normalization metric of this error controller, the allowable error scale for the temperature component is about 100 K. For trace species, the absolute tolerance dominates; for example, with absolute and relative tolerances of 10^{-8} and 10^{-4} respectively, and a species mass fraction of 10^{-6}, the contribution of the relative tolerance is negligible (its contribution is 100 times smaller than the absolute tolerance). In this case, the estimated error must be below 10^{-8} to be considered acceptable. After estimating the error, OpenFOAM uses a controller to adaptively adjust the step size to ensure computational efficiency. In practice, because three-dimensional turbulent combustion simulations involve numerous error sources, the absolute and relative tolerances of the ODE solver are not set very strictly, so as to avoid wasting computational resources.

1.3.5. OpenFOAM Implementation Details

1.3.5.1. Technical Style

The following are personal opinions. OpenFOAM-10 currently provides explicit methods, linearized implicit methods, and extrapolation methods based on the linearized implicit method (such as `seulex`). Compared with Chemkin or Cantera, OpenFOAM has the following characteristics that make it more suitable as an engineering application tool:

1. The linearized implicit method avoids the Newton iteration solving required by fully implicit methods, thereby also bypassing the convergence and initial-value sensitivity problems of the iteration process, improving engineering applicability.
2. The linearized implicit method (e.g., `Rosenbrock`) imposes extremely strict requirements on the accuracy of the Jacobian matrix, since the Jacobian itself is part of the computational scheme. If an imprecise Jacobian approximation is used, it will lead to loss of accuracy and may destroy its stability properties.
3. When the reaction mechanism is small, the computational cost of an exact analytical Jacobian matrix is acceptable; in this case the linearized implicit method can improve computational efficiency while guaranteeing accuracy, and effectively avoid the initial-value sensitivity problem of Newton iteration, demonstrating good engineering applicability.

In contrast, software such as Cantera or Chemkin typically uses implicit multi-step solvers such as CVODE/BDF, and generally has the following technical characteristics that make it more suitable as a reaction kinetics research tool:

1. Using approximate Jacobian matrices combined with sparse storage formats reduces memory footprint and computational overhead, which is especially suitable for large-scale reaction mechanisms; however, approximate Jacobian matrices are unfriendly to the linearized implicit method, affecting order, stability, or efficiency.
2. Using fully implicit multi-step methods, the requirement on the accuracy of the Jacobian matrix is not very strict, because at this point the Jacobian matrix only provides a descent direction for the Newton iteration process; however, the iteration process itself also faces initial-value sensitivity problems, so the Jacobian matrix cannot be too inaccurate, to avoid a wrong descent direction.

3. The BDF method requires multiple previous sub-time steps, hence the concept of restarting is involved. Therefore, for three-dimensional turbulent combustion numerical simulation, the efficiency of the BDF method may decrease as the CFD time step decreases, because after each CFD time step update, the ODE integration must be restarted from a new initial state, and the multi-step method cannot exploit historical information across CFD time steps. However, for steady-state solutions or zero-dimensional ignition problems, excessive restarting is not involved.

1.3.5.2. Seulex

The Seulex method, fully named **Semi-implicit Euler Extrapolation**, is a numerical method for solving stiff systems of ordinary differential equations. Its core idea is: use the linearized implicit Euler method to generate a series of low-accuracy solutions at different step sizes, and then combine these solutions through polynomial extrapolation to construct a high-accuracy approximate solution. This method possesses both **adaptive step size** and **order control** capabilities, is suitable for stiff problems in engineering computation, and is widely used in CFD software such as OpenFOAM.

The Seulex solver in the OpenFOAM Foundation version has undergone multiple modifications. In versions before OpenFOAM-5.x, the denominator used for the monotonicity check was computed as follows:

```
            const scalar denom = min(1, dy1 + SMALL);
```

In the OpenFOAM-5.x version, the code was modified to:

```
            const scalar denom = max(1, dy1);
```

From OpenFOAM-6 to OpenFOAM-13, the code was modified to:

```
            const scalar denom = min(1, dy1 + small);
```

In OpenFOAM-14, the code was modified again, and there is also another modified location:

```
            const scalar denom = max(1, dy1);
            ...
            //errOld = min(4*err, 1); this line of code is from versions before OpenFOAM-14
            errOld = max(4*err, 1);
```

The table below lists the **number of ODE time steps** required to compute zero-dimensional ignition cases using the Seulex code from different OpenFOAM versions. The first column is the reaction mechanism, and the first row indicates absolute and relative tolerances of 10^{-8} and 10^{-2}, and 10^{-12} and 10^{-6}, respectively; the second row indicates the corresponding OpenFOAM version. It can be seen that under stricter convergence tolerances, the Seulex code of OF14 basically requires the fewest number of ODE time steps. Overall, under the zero-dimensional ignition cases and tolerance settings tested in this paper, the Seulex implementation of the OF14 version generally exhibits fewer ODE time steps.

	A8R2	A8R2	A8R2	A12R6	A12R6	A12R6
	OF5x	OF10	OF14	OF5x	OF10	OF14
C2H4_59s_271r	62	112	55	1250	3155	111
C2H4_25s_70r	62	111	44	1872	4843	110

	A8R2	A8R2	A8R2	A12R6	A12R6	A12R6
C2H5OH_57s_383r	66	118	46	1607	4051	122
C2H5OH_39s_370r	77	115	75	1849	4971	120
C3H8_70s_463r	42	131	42	1637	4812	50
CH3OH_116s_962r	44	94	39	1171	4239	83
CH4_21s_84r	1035	1057	1028	2013	3782	1064
CH4_28s_116r	1033	1053	1034	1819	2878	1058
CH4_38s_291r	1219	1261	1215	2194	3797	1267
CH4_GRI30_53s_325r	1033	1063	1015	1964	3872	1064
CH4_68s_631r	1030	1058	1018	1976	4705	1058
CH4_59s_271r	1032	1065	1021	1975	3988	1067
DME_26s_71r	152	209	149	2321	5601	231
DME_79s_351r	144	221	146	2123	6672	212
DME_39s_175r	162	224	153	2397	6172	224
H2_13s_27r	50	56	50	749	1718	71
H2_10s_21r	52	57	52	692	1488	72
H2_15s_71r	67	72	55	824	1785	85
IC8H18_140s_643r	436	513	439	2207	7808	516
Kerosene_28s_92r	5037	5078	5014	6270	8277	5067
Kerosene_32s_77r	2068	2113	2055	4288	9314	2145
NC12H26_31s_193r	1041	1086	1038	2504	4870	1079
NC12H26_54s_269r	1038	1083	1028	2299	4911	1090
NC7H16_46s_115r	80	131	77	1571	4722	140
NC7H16_68s_283r	88	142	79	1723	5571	149
NC7H16_32s_104r	361	398	352	1983	4819	412
NH3CH4_129s_1231r	1057	1112	1055	2179	4650	1111
NH3CH4_59s_356r	1048	1099	1029	2326	3955	1078
NH3CH4_42s_130r	1059	1100	1036	2570	4327	1085
NH3CH4_125s_1099r	1043	1093	1037	2265	5053	1086
NH3_38s_232r	38	112	41	1321	3536	143
NH3_32s_213r	67	89	58	1258	3469	110
NH3_31s_200r	351	364	328	1385	3513	369
NH3_38s_263r	37	96	38	1326	3541	114

1.3.5.3. ODE Solver Tolerance Inconsistency

OpenFOAM-10 and later versions use mass fraction as the state variable of the chemical reaction ODE system, while OpenFOAM-9 and earlier versions used molar concentration. Because mass fraction and molar concentration differ significantly in numerical magnitude (the former is dimensionless with its components summing to 1, while the latter is affected by temperature and pressure and can span multiple orders of magnitude), **the same absolute tolerance and relative tolerance settings do not represent the same computational accuracy or efficiency under these two state variables**. The OpenFOAM Foundation versions do not provide compatibility handling for this difference, nor do they provide an automatic conversion mechanism for state-variable migration.

This solver supports both mole number and mass fraction as state variables, and the user can freely choose at runtime. When mole number is selected as the state variable, the direct result of the ODE solver integration is N^{N+1}. Before entering the error estimation and step-size control module, the solver first converts the mole number into its corresponding mass fraction, and then computes the normalized error based on the mass fraction. The reason for this is: the absolute and relative tolerances in the error controller are designed for the physical magnitude of the mass fraction, and directly performing error evaluation on the mole number would render the tolerance settings physically meaningless (the mole numbers of different species can differ by several orders of magnitude). The conversion formula from mole number to mass fraction is:

$$Y_i = \frac{N_i W_i}{\sum_{j=1}^{N_s} N_j W_j}, \qquad i = 1, 2, \dots, N_s \tag{27}$$

1.4. Jacobian Matrix

The linearized implicit method generally requires the Jacobian matrix to provide linearization information; the Jacobian matrix can be computed via numerical differencing or by directly providing analytical expressions, and the accuracy of the Jacobian matrix has a considerable impact on the convergence rate of the ODE solver. This chapter introduces the Jacobian matrices for different state variables and discusses related technical details. The Jacobian matrices involved in this chapter are expanded only to the level of the thermophysical model, not to the level of the reaction kinetics model.

When taking partial derivatives, ensure that the other variables of the multivariate function are fixed as constants, and further expand the intermediate functions. In addition, the Jacobian matrix here is obtained by treating all species as independent variables. Note that "treating all species as independent variables" is a numerical choice; physically, one species is not an independent species.

The derivations below are based on the adiabatic constant-pressure condition and the ideal gas.

1.4.1. Species Mass Fraction

- Partial derivative of the mass fraction rate of change with respect to the mass fraction

Taking the partial derivative with respect to the j-th species mass fraction on both sides of the above expression, **with the other species and the temperature and pressure treated as constants**. Here the density ρ and the reaction rate $\dot{\omega}_i$ are functions of Y_j, so we have:

$$\frac{d}{dY_j}\left(\frac{dY_i}{dt}\right) = \frac{W_i}{\rho}\frac{\partial \dot{\omega}_i}{\partial Y_j} - \frac{dY_i}{dt}\frac{1}{\rho}\frac{\partial \rho}{\partial Y_j} \tag{28}$$

Here $\frac{\partial \rho}{\partial Y_j}$ needs further expansion and is omitted here. The above expression is the submatrix of partial derivatives of the species mass fraction rate of change with respect to the species mass fraction in the Jacobian matrix, of size $Ns \times Ns$. The reaction rate $\dot{\omega}_i$ requires molar concentration as an intermediate variable, so by the chain rule we have:

$$\frac{\partial \dot{\omega}_i}{\partial Y_j} = \sum_{k=1}^{N_s} \frac{\partial \dot{\omega}_i}{\partial C_k}\frac{\partial C_k}{\partial Y_j} \tag{29}$$

The above expression is a standard matrix multiplication; on the right are, respectively, the partial derivative matrix of the reaction rate with respect to molar concentration and the partial derivative matrix of molar concentration with respect to species mass fraction. The analytical expression of $\frac{\partial \dot{\omega}_i}{\partial C_k}$ requires expansion down into the reaction kinetics model, while $\frac{\partial C_k}{\partial Y_j}$ can be further expanded:

$$\frac{\partial C_i}{\partial Y_j} = \frac{\partial}{\partial Y_j}\left(\frac{\rho Y_i}{W_i}\right) = \frac{\partial \rho}{\partial Y_j}\frac{Y_i}{W_i} + \frac{\rho}{W_i}\frac{\partial Y_i}{\partial Y_j} = \frac{\partial \rho}{\partial Y_j}\frac{Y_i}{W_i} + \frac{\rho}{W_i}\delta_{ij} \tag{30}$$

Here δ_{ij} is 1 when $i = j$, and 0 otherwise.

- Partial derivative of the mass fraction rate of change with respect to temperature

$$\frac{\partial}{\partial T}\left(\frac{dY_i}{dt}\right) = \frac{1}{T}\left(\frac{dY_i}{dt}\right) + \frac{W_i}{\rho}\left(-\sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_i}{\partial C_k}\frac{C_k}{T} + \frac{\partial \dot{\omega}_i}{\partial T}\right) \tag{31}$$

- Partial derivative of the temperature rate of change with respect to the mass fraction

$$\frac{\partial}{\partial Y_i}\left(\frac{dT}{dt}\right) = -\left[\frac{Cp_i}{Cp_m}\frac{dT}{dt} + \frac{1}{Cp_m}\sum_{j=1}^{N_s} Ha_j \frac{\partial}{\partial Y_i}\left(\frac{dY_j}{dt}\right)\right] \tag{32}$$

- Partial derivative of the temperature rate of change with respect to temperature

$$\frac{\partial}{\partial T}\left(\frac{dT}{dt}\right) = \frac{1}{Cp_m}\left[-\frac{\partial Cp_m}{\partial T}\frac{dT}{dt} - \sum_{i=1}^{N_s} Cp_i \frac{dY_i}{dt} - \sum_{i=1}^{N_s} Ha_i \frac{\partial}{\partial T}\left(\frac{dY_i}{dt}\right)\right] \tag{33}$$

In the above formulas, Cp_i is the constant-pressure specific heat capacity per unit mass of the i-th species ($\mathrm{J/kg/K}$), and Cp_m is the mixture constant-pressure specific heat capacity per unit mass. Ha_j is the specific absolute enthalpy per unit mass of the j-th species ($\mathrm{J/kg}$).

1.4.2. Molar Concentration

- The partial derivative of the molar concentration rate of change with respect to molar concentration is as follows:

$$\begin{aligned}\frac{\partial}{\partial C_j}\frac{dC_i}{dt} = \frac{\partial \dot{\omega}_i}{\partial C_j} - \delta_{ij}\left(\frac{1}{C_{\mathrm{tot}}}\sum_{k=1}^{N_s}\dot{\omega}_k + \frac{1}{T}\frac{dT}{dt}\right) \\ + C_i\left[\frac{1}{C_{\mathrm{tot}}^2}\sum_{k=1}^{N_s}\dot{\omega}_k - \frac{1}{C_{\mathrm{tot}}}\sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_k}{\partial C_j} - \frac{1}{T}\frac{\partial}{\partial C_j}\frac{dT}{dt}\right]\end{aligned} \tag{34}$$

- The partial derivative of the temperature rate of change with respect to molar concentration is as follows, where Cp_{vol} is the constant-pressure volumetric heat capacity ($\mathrm{J/m^3/K}$).

$$\frac{\partial}{\partial C_i}\frac{dT}{dt} = -\frac{1}{Cp_{vol}}\left(cp_i\frac{dT}{dt} + \sum_{j=1}^{N_s} ha_j \frac{\partial \dot{\omega}_j}{\partial C_i}\right) \tag{35}$$

- The partial derivative of the molar concentration rate of change with respect to temperature is as follows:

$$\frac{\partial}{\partial T}\frac{dC_i}{dt} = \frac{\partial \dot{\omega}_i}{\partial T} - \frac{C_i}{C_{tot}}\sum_{j=1}^{N_s}\frac{\partial \dot{\omega}_j}{\partial T} - \frac{C_i}{T}\frac{\partial}{\partial T}\frac{dT}{dt} + \frac{C_i}{T^2}\frac{dT}{dt} \tag{36}$$

- The partial derivative of the temperature rate of change with respect to temperature is as follows:

$$\frac{\partial}{\partial T}\frac{dT}{dt} = \frac{1}{Cp_{vol}}\left[-\frac{\partial Cp_{vol}}{\partial T}\frac{dT}{dt} - \sum_{i=1}^{N_s} cp_i\,\dot{\omega}_i - \sum_{i=1}^{N_s} ha_i\frac{\partial \dot{\omega}_i}{\partial T}\right] \tag{37}$$

In the above formulas, cp_i is the constant-pressure molar heat capacity of the i-th species ($\mathrm{J/kmol/K}$), and Cp_{vol} is the mixture constant-pressure volumetric heat capacity ($\mathrm{J/m^3/K}$). ha_j is the specific molar absolute enthalpy of the j-th species ($\mathrm{J/kmol}$).

1.4.3. Mole Number

- The partial derivative of the species mole-number rate of change with respect to mole number is:

$$\frac{\partial}{\partial N_j}\left(\frac{dN_i}{dt}\right) = \left(\dot{\omega}_i - \sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_i}{\partial C_k}C_k\right)\frac{V}{N_{tot}} + \frac{\partial \dot{\omega}_i}{\partial C_j} \tag{38}$$

- The partial derivative of the species mole-number rate of change with respect to temperature is:

$$\frac{\partial}{\partial T}\left(\frac{dN_i}{dt}\right) = \frac{\partial}{\partial T}(\dot{\omega}_i V) = V\left(-\frac{1}{T}\sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_i}{\partial C_k}C_k + \frac{\partial \dot{\omega}_i}{\partial T}\right) + \frac{1}{T}\frac{\partial N_i}{\partial t} \tag{39}$$

- The partial derivative of the temperature rate of change with respect to temperature is:

$$\frac{\partial}{\partial T}\frac{dT}{dt} = \frac{1}{Cp_{\mathrm{tot}}}\left(-\frac{\partial Cp_{\mathrm{tot}}}{\partial T}\frac{dT}{dt} - \sum_{i=1}^{N_s} cp_i\frac{dN_i}{dt} - \sum_{i=1}^{N_s} ha_i\frac{d}{dT}\left(\frac{dN_i}{dt}\right)\right) \tag{40}$$

- The partial derivative of the temperature rate of change with respect to the species mole number is:

$$\frac{\partial}{\partial N_i}\frac{dT}{dt} = -\left(cp_i\frac{dT}{dt} + \sum_{j=1}^{N_s} ha_j\frac{\partial}{\partial N_i}\left(\frac{dN_j}{dt}\right)\right)\frac{1}{Cp_{\mathrm{tot}}} \tag{41}$$

In the above formulas, Cp_{tot} is the constant-pressure heat capacity ($\mathrm{J/K}$).

1.4.4. OpenFOAM Implementation Details

1.4.4.1. Computation of the Jacobian Matrix

In OpenFOAM-5.x and earlier versions, the computation of the Jacobian matrix was rather crude: the partial derivative of the concentration rate of change with respect to temperature, $\frac{\partial}{\partial T}\frac{dC_i}{dt}$, within the Jacobian matrix was obtained via numerical differencing. At the same time, the partial derivatives of the temperature rate of change with respect to concentration and temperature were apparently neglected, as were certain partial derivatives in some reaction kinetics models. The code is as follows:

```
template<class CompType, class ThermoType>
void Foam::chemistryModel<CompType, ThermoType>::Jacobian
(
```

```
    const scalar t,
    const scalarField& c,
    scalarField& dcdt,
    scalarSquareMatrix& dfdc
) const
{
    ...
    // Calculate the dcdT elements numerically
    const scalar delta = 1.0e-3;

    omega(c_, T + delta, p, dcdt_);
    for (label i=0; i<nSpecie_; i++)
    {
        dfdc(i, nSpecie_) = dcdt_[i];
    }

    omega(c_, T - delta, p, dcdt_);
    for (label i=0; i<nSpecie_; i++)
    {
        dfdc(i, nSpecie_) = 0.5*(dfdc(i, nSpecie_) - dcdt_[i])/delta;
    }

    dfdc(nSpecie_, nSpecie_) = 0;
    dfdc(nSpecie_ + 1, nSpecie_) = 0;
}
```

Starting with OpenFOAM-6, OpenFOAM provides an analytical Jacobian matrix; however, it should be noted that this version still neglects the effect of the volume-change term when computing the molar concentration rate of change. Consequently, the Jacobian matrix was derived under the assumption of constant-volume conditions ($\frac{dC_i}{dt} = \dot{\omega}_i$). Thus, for OpenFOAM-6 through OpenFOAM-9, the partial derivatives of the molar concentration rate of change with respect to molar concentration and temperature are, respectively:

$$\begin{aligned} \frac{\partial}{\partial C_j}\frac{dC_i}{dt} &= \frac{\partial \dot{\omega}_i}{\partial C_j} \\ \frac{\partial}{\partial T}\frac{dC_i}{dt} &= \frac{\partial \dot{\omega}_i}{\partial T} \end{aligned} \tag{42}$$

it should be noted that although the **mathematical derivation** is correct, the **physical premise** is wrong: it is derived from the molar concentration rate-of-change expression under constant-volume conditions, whereas what we actually need is the expression under constant-pressure conditions.

Another issue is that, in OpenFOAM-6 through OpenFOAM-8, an erroneous code snippet `J(nSpecie_, nSpecie_) += dTdt/T;` was provided for the partial derivative of the temperature rate of change with respect to temperature; this code is redundant and should be removed.

```
template<class ReactionThermo, class ThermoType>
void Foam::StandardChemistryModel<ReactionThermo, ThermoType>::Jacobian
(
    const scalar t,
    const scalarField& c,
    const label li,
    scalarField& dcdt,
```

```
    scalarSquareMatrix& J
) const
{
    ...
    // ddT of dTdt
    J(nSpecie_, nSpecie_) = 0;
    for (label i = 0; i < nSpecie_; i++)
    {
        J(nSpecie_, nSpecie_) += cpi[i]*dcdt[i] + hi[i]*J(i, nSpecie_);
    }
    J(nSpecie_, nSpecie_) += dTdt*dcpdTMean;
    J(nSpecie_, nSpecie_) /= -cpMean;
    J(nSpecie_, nSpecie_) += dTdt/T;// this code is redundant
}
```

This error was fixed in OpenFOAM-9, as shown in the following code:

```
template<class ThermoType>
void Foam::standardChemistryModel<ThermoType>::Jacobian
(
    const scalar t,
    const scalarField& c,
    const label li,
    scalarField& dcdt,
    scalarSquareMatrix& J
) const
{
    ...
    // d(dTdt)/dT
    scalar& d2TdtdT = J(nSpecie_, nSpecie_);
    for (label i = 0; i < nSpecie_; i++)
    {
        const scalar d2cidtdT = J(i, nSpecie_);
        d2TdtdT -=
            dcdt[i]*specieThermos_[i].cp(p, T)
          + d2cidtdT*specieThermos_[i].ha(p, T);
    }
    d2TdtdT -= dTdt*dccpdT;
    d2TdtdT /= ccp;

    // d(dpdt)/dc = 0 (pressure is assumed constant)

    // d(dpdt)/dT = 0 (pressure is assumed constant)
}
```

Starting with OpenFOAM-10, the state variables are no longer molar concentrations but mass fractions, so the Jacobian matrix also takes the form based on mass fractions, and the Jacobian matrix is analytical. However, the mass-fraction-form Jacobian matrix is computationally expensive. To save computational cost, OpenFOAM-10 by default simplifies the computation of the partial-derivative matrix of molar concentration with respect to mass fraction, reducing the overall computational time complexity from cubic to quadratic in the number of species. The specific expressions are as follows; it can be seen that when the `fast` option is used, only the

diagonal terms of the partial-derivative matrix of molar concentration with respect to mass fraction are retained.

$$\begin{aligned} \frac{\partial C_i}{\partial Y_j} &= \frac{\partial \rho}{\partial Y_j}\frac{Y_i}{W_i} + \frac{\rho}{W_i}\delta_{ij} \qquad \text{(exact jacobian)} \\ \frac{\partial C_i}{\partial Y_j} &= \frac{\rho}{W_i}\delta_{ij} \qquad \text{(fast jacobian)} \end{aligned} \tag{43}$$

The specific implementation is shown in the following code block.

```
template<class ThermoType>
void Foam::chemistryModel<ThermoType>::Jacobian
(
    const scalar t,
    const scalarField& YTp,
    const label li,
    scalarField& dYTpdt,
    scalarSquareMatrix& J
) const
{
    ...
    // Evaluate the derivatives of concentration w.r.t. mass fraction
    scalarSquareMatrix& dcdY = YTpYTpWork_[0];
    for (label i=0; i<nSpecie_; i++)
    {
        const scalar rhoMByWi = rhoM/specieThermos_[sToc(i)].W();
        switch (JacobianType_)
        {
            case JacobianType::fast:
                {
                    dcdY(i, i) = rhoMByWi;
                }
                break;
            case JacobianType::exact:
                for (label j=0; j<nSpecie_; j++)
                {
                    dcdY(i, j) =
                        rhoMByWi*((i == j) - rhoM*v[sToc(j)]*Y_[sToc(i)]);
                }
                break;
        }
    }
    ...
    ...
    for (label i=0; i<nSpecie_; i++)
    {
        ...
        for (label j=0; j<nSpecie_; j++)
        {
            scalar ddNidtByVdYj = 0;
            switch (JacobianType_)
            {
```

```
                case JacobianType::fast:
                    {
                        const scalar ddNidtByVdcj = ddNdtByVdcTp(i, j);
                        ddNidtByVdYj = ddNidtByVdcj*dcdY(j, j);
                    }
                    break;
                case JacobianType::exact:
                    for (label k=0; k<nSpecie_; k++)
                    {
                        const scalar ddNidtByVdck = ddNdtByVdcTp(i, k);
                        ddNidtByVdYj += ddNidtByVdck*dcdY(k, j);
                    }
                    break;
            }

            scalar& ddYidtdYj = J(i, j);
            ddYidtdYj = WiByrhoM*ddNidtByVdYj + rhoM*v[sToc(j)]*dYidt;
        }
        ...
    }
}
```

1.4.4.2. Effect of the dCdY Approximation on Computational Efficiency

OpenFOAM-10 offers two ways to compute the partial-derivative matrix of molar concentration with respect to mass fraction, namely approximate and analytical computation; accordingly, the resulting Jacobian matrix is also divided into approximate and analytical Jacobian matrices. Although the approximate Jacobian matrix reduces the computational cost of the Jacobian matrix, because the approximate Jacobian degrades the accuracy of the local linearization, it may lead to larger error estimates, more conservative step-size control, or more rejected time steps, thereby increasing the total number of ODE steps. Whether a gain in computational efficiency is achieved also depends on the specific situation. This subsection tests the effect of the analytical and approximate Jacobian matrices on computational efficiency, using the zero-dimensional ignition test case under the project folder.

When compiling the code, ensure that the following macros are defined as `true`, so that the **Jacobian partial derivatives of pressure-dependent reactions (third-body and chemically-activated reactions) in the reaction kinetics model** are computed exactly, rather than being neglected or approximated:

```
#define FALL_OFF_FUNCTION_JACOBIAN true
#define CHEMICALLY_ACTIVATION_FUNCTION_JACOBIAN true
```

The results of the zero-dimensional ignition test case are given below, using the Rodas34 solver. The first column is the reaction mechanism; the second column gives the total number of ODE sub-time steps invoked by each of the two Jacobian computation methods over the entire zero-dimensional ignition integration process, when the absolute and relative tolerances are 10^{-12} and 10^{-6}, respectively. For example, for the C2H4_Sandiego case, the required number of ODE time steps when using the approximate Jacobian matrix is more than ten times that of the analytical Jacobian matrix. When using 10^{-8} and 10^{-2}, however, the difference in ODE time steps is very small.

From an engineering-practice perspective, the Exact mode is recommended when using small mechanisms and tight tolerances, and the Fast mode otherwise. Although the Fast mode sacrifices some accuracy, given the numerous sources of error in CFD, this loss of accuracy is likely far outweighed by other potential large error sources (mesh, turbulence model, combustion model).

	A12R6	**A12R6**	**A8R2**	**A8R2**
	Exact	**Fast**	**Exact**	**Fast**
C2H4_59s_271r	1435	17119	80	84
C2H4_25s70r	1165	23192	49	79
C2H5OH_57s_383r	1535	20895	74	77
C2H5OH_39s_370r	1694	31300	76	84
C3H8_70s_463r	3357	18765	49	87
CH3OH_116s_962r	1305	34251	59	64
CH4_21s_84r	2007	9500	1037	1039
CH4_28s_116r	1784	10505	1035	1040
CH4_38s_291r	2212	14300	1220	1240
CH4_53s_325r	2031	11468	1039	1042
CH4_68s_631r	2211	18920	1036	1028
CH4_59s_271r	2202	13546	1041	1044
DME_26s_71r	1786	27214	166	175
DME_79s_351r	2545	39976	172	176
DME_39s_175r	2003	35887	174	178
H2_13s_27r	458	17731	45	50
H2_10s_21r	432	15404	47	55
H2_15s_71r	512	16749	61	70
IC8H18_140s_643r	2678	35402	465	466
Kerosene_28s_92r	6103	22224	5046	5047
Kerosene_32s_77r	4208	35167	2082	2093
NC12H26_31s_193r	2275	21744	1050	1051
NC12H26_54s_269r	2692	33674	1049	1052
NC7H16_46s_115r	1501	27281	95	97
NC7H16_68s_283r	1780	35718	99	104
NC7H16_32s_104r	1969	28562	361	366
NH3CH4_129s_1231r	2139	26165	1073	1074
NH3CH4_59s_356r	2094	9206	1056	1057

	A12R6	A12R6	A8R2	A8R2
NH3CH4_42s_130r	2273	11745	1061	1063
NH3CH4_125s_1099r	2216	12397	1060	1061
NH3_38s_232r	2812	10242	50	51
NH3_32s_213r	1637	13624	69	72
NH3_31s_200r	1232	13327	322	333
NH3_38s_263r	1066	10158	64	64

1.4.4.3. Number of Independent Species

OpenFOAM selects all species as independent variables entering the ordinary differential equation system. Some solvers (e.g., pyJac) use Ns-1 species as independent variables, with the last species treated as a dependent variable obtained through a constraint condition; for example, for species mass fractions, the last species can be obtained via the following constraint condition:

$$Y_{N_s} = 1 - \sum_{i=1}^{N_s-1} Y_i \tag{44}$$

For molar concentration and number of moles, the ideal gas equation of state can be used to obtain the molar concentration and number of moles of the last species:

$$C_{N_s} = C_{\text{tot}} - \sum_{i=1}^{N_s-1} C_i = \frac{P}{RuT} - \sum_{i=1}^{N_s-1} C_i \tag{45}$$

The expression for the number of moles is:

$$N_{N_s} = N_{\text{tot}} - \sum_{i=1}^{N_s-1} N_i = \frac{PV}{R_u T} - \sum_{i=1}^{N_s-1} N_i \tag{46}$$

The choice of state variables above does not affect the computation of the reaction rate, but it does affect the operations on the partial-derivative matrix. For example, the partial derivative of the mixture density with respect to the species mass fraction (with the other species mass fractions, temperature, and pressure held constant) yields different expressions depending on whether Ns or Ns-1 independent species are chosen as state variables:

$$\frac{\partial \rho}{\partial Y_i} = \frac{P}{RuT}\frac{\partial W_m}{\partial Y_i} = -\frac{P}{RuT}\frac{{W_m}^2}{W_i}, \qquad i = 1, 2, \dots, N_s \tag{47}$$

For the case of Ns-1 independent species:

$$\frac{\partial \rho}{\partial Y_i} = -\frac{P}{R_u T}{W_m}^2 \left(\frac{1}{W_i} - \frac{1}{W_{N_s}} \right), \qquad i = 1, 2, \dots, N_s - 1 \tag{48}$$

These two state-variable selection schemes are mathematically equivalent, but they may affect the sparsity and condition number of the Jacobian matrix, and thus the convergence rate and stability of the ODE solver. This project is consistent with OpenFOAM, choosing all species as state variables.

1.5. Reaction Kinetics Model

Combustion numerical simulation mainly relies on macroscopic-scale reaction kinetics models, and typically does not involve mesoscopic or microscopic scales. Its core consists of the law of mass action and the Arrhenius law, which describe the dependence of the reaction rate on concentration and temperature, respectively.

1.5.1. Standard Arrhenius Reaction

Consider the r-th reaction involving Ns species; its stoichiometric equation can be written as:

$$\sum_{k=1}^{N_s} \nu'_{k,r} M_k \rightleftharpoons \sum_{k=1}^{N_s} \nu''_{k,r} M_k \qquad (r = 1, 2, \cdots, Nr) \tag{50}$$

where M_k denotes the molecular formula of the k-th species, and $\nu'_{k,r}$ and $\nu''_{k,r}$ are the forward and reverse stoichiometric coefficients of the k-th species in the r-th reaction (dimensionless integers, both denoted by the italic ν). The net stoichiometric coefficient of the k-th species in the r-th reaction is defined as:

$$\nu_{k,r} = \nu''_{k,r} - \nu'_{k,r} \tag{51}$$

Let q denote the reaction rate; then the chemical reaction rate of the i-th species is:

$$\dot{\omega}_i = \sum_{r=1}^{N_r} \nu_{ri} q_r \tag{52}$$

For a standard Arrhenius chemical reaction, the reaction rate is determined by the rate constant and the concentration product:

$$q = k_f C_f - k_r C_r = k_f \prod_{i=1}^{N_s} C_i^{\nu'_i} - k_r \prod_{i=1}^{N_s} C_i^{\nu''_i} \tag{53}$$

where k_f is the forward rate constant, C_f is the reactant concentration product, k_r is the reverse rate constant, and C_r is the product concentration product; the forward rate constant is determined by the Arrhenius model:

$$k_f(T) = A\, T^{\beta} \exp\left(-\frac{Ea}{RuT}\right) \tag{54}$$

where A is the pre-exponential factor, β is the temperature exponent, Ea is the activation energy, Ru is the universal gas constant, and T is the temperature. The reverse reaction rate constant is determined jointly by the chemical equilibrium constant K_c and the forward reaction rate constant k_f:

$$k_r = \frac{k_f}{K_c} \tag{55}$$

$$K_c = K_p \cdot \left(\frac{P_{std}}{RuT}\right)^{\sum_{k=1}^{Ns} \nu_k} \tag{56}$$

where P_{std} denotes the standard pressure and $\sum_{k=1}^{Ns} \nu_k$ denotes the sum of the stoichiometric numbers of this reaction. For example, for the reaction A+B=C, $\sum_{k=1}^{Ns} \nu_k = 1 - 1 - 1 = -1$; for the reaction A+1.5B=C, $\sum_{k=1}^{Ns} \nu_k = 1 - 1.5 - 1 = -1.5$. K_p is the partial-pressure equilibrium constant, whose expression is:

$$K_p = \exp\left(-\frac{\Delta G_{std}}{RuT}\right) \tag{57}$$

where G_{std} is the standard Gibbs free energy `(J/kmol)`, and ΔG_{std} denotes the change in the standard Gibbs free energy resulting from the complete conversion of reactants into products.

1.5.2. Third-Body-Enhanced Reaction

The reaction rate of a third-body-enhanced reaction is:

$$q = \left(\sum_{i=1}^{Ns} a_i C_i\right)(k_f C_f - k_r C_r) \tag{58}$$

where a_i is the third-body efficiency factor. Let $M = \sum_{i=1}^{Ns} a_i C_i$ denote the third-body efficiency.

1.5.3. Pressure-Dependent Reaction

This subsection introduces pressure-dependent reactions, including Fall off reactions, Chemically activated reactions and pressure-dependent logarithmic (PLOG) reactions.

For a Falloff reaction, its form is:

$$k_f = k_\infty \frac{Pr}{1 + Pr} F \tag{59}$$

The form of a ChemicallyActivated reaction is:

$$k_f = k_0 \frac{1}{1 + Pr} F \tag{60}$$

where

$$Pr = \frac{k_0 M}{k_\infty} \tag{61}$$

Both rate constants take the Arrhenius form:

$$\begin{aligned} k_0 &= A_0 T^{\beta_0} \exp(-Ea_0/RuT) \\ k_\infty &= A_\infty T^{\beta_\infty} \exp(-Ea_\infty/RuT) \end{aligned} \tag{62}$$

For the computation of the factor F, there are mainly three forms; when the Lindemann form is used, $F = 1$.

Using the Troe form gives:

$$\log_{10} F = \left[1 + \left[\frac{\log_{10} Pr + c}{n - d(\log_{10} Pr + c)}\right]^2\right]^{-1} \log_{10} F_{cent} \tag{63}$$

where $\log_{10}$ is the base-10 logarithm.

$$c = -0.4 - 0.67 \log F_{cent} \tag{64}$$

$$n = 0.75 - 1.27 \log F_{cent} \tag{65}$$

$$d = 0.14 \tag{66}$$

$$F_{cent} = (1-\alpha)\exp(-T/T^{***}) + \alpha \exp(-T/T^{*}) + \exp(-T^{**}/T) \tag{67}$$

where α, T^*, T^{**}, and T^{***} need to be supplied by the reaction mechanism file.

For the SRI form, we have:

$$\begin{aligned} F &= d\left[a \cdot \exp\left(\frac{-b}{T}\right) + \exp\left(\frac{-T}{c}\right)\right]^X T^e \\ X &= \frac{1}{1+(\log Pr)^2} \end{aligned} \tag{68}$$

where a, b, c, d, e are the input parameters of the SRI form, provided by the reaction mechanism file.

In addition to Falloff reactions, PLOG (pressure-logarithmic) forms are also frequently used in combustion reaction mechanisms to describe pressure-dependent reactions. A PLOG reaction directly gives the corresponding Arrhenius parameters at several specified pressures, and the rate constant at the target pressure is then obtained by interpolation.

For a PLOG reaction, if its rate constant at the j-th reference pressure P_j is denoted by $k_{f,j}$, then:

$$k_{f,j}(T) = A_j T^{\beta_j} \exp\left(-\frac{E_{a,j}}{R_u T}\right) \tag{69}$$

where A_j, β_j, and $E_{a,j}$ denote the pre-exponential factor, temperature exponent, and activation energy of this reaction at pressure P_j, respectively. If the current pressure P lies between two discrete pressure points P_j and P_{j+1}, the following is typically adopted:

$$\log k_f(P,T) = \log k_{f,k}(T) + \frac{\log P - \log P_k}{\log P_{k+1} - \log P_k}[\log k_{f,k+1}(T) - \log k_{f,k}(T)] \tag{70}$$

The reverse rate constant is still determined by the chemical equilibrium constant. To date, OpenFOAM does not support PLOG reactions; this solver provides the relevant support.

1.5.4. Unit Conversion

OpenFOAM typically uses the International System of Units (m^3,kmol,s,J), whereas reaction kinetics software such as Chemkin typically uses (cm^3,mol,s,cal) as units.

The unit conversion of the pre-exponential factor A is introduced below. It should be noted that the unit of the reaction rate q obtained from the Arrhenius model and the law of mass action is always **concentration/time**; therefore, the unit of the rate constant must be deduced backward from the unit of the concentration product obtained from the law of mass action.

Let ν be the sum of the forward stoichiometric numbers of this reaction; then the unit of the concentration product is concentration raised to the power ν. Clearly, the unit of the rate constant should be concentration to the power $1-\nu$ divided by time. Therefore, when converting the unit of the reaction rate from $\mathrm{mol/cm}^3/\mathrm{s}$ to $\mathrm{kmol/m}^3/\mathrm{s}$, the following should hold:

$$mol/cm^3/s = [(mol/cm^3)^{(1-\nu)}/s][(mol/cm^3)^{\nu}] \tag{71}$$

The first box on the right-hand side of the expression is the unit of the rate constant, and the second box is the unit of the concentration product. What we need to care about is the unit of the rate constant, because the rate constants in the reaction mechanism file are stored in units of (cm^3, mol, s, cal). In addition, T^{β} is regarded as dimensionless, and the numerator and denominator inside the exponent become dimensionless after division; therefore, the only unit that ultimately needs to be determined is that of the pre-exponential factor. Substituting the unit conversions for length and amount of substance gives:

$$(mol/cm^3)^{(1-\nu)}/s = (kmol/m^3 \frac{1000000}{1000})^{(1-\nu)}/s \tag{72}$$

Therefore, the scaling factor of the pre-exponential factor is obtained as $1000^{(1-\nu)}$.

The unit of activation energy is simpler: if the activation energy in Chemkin is in cal/mol, it can be converted to the standard unit of $4184 J/kmol$. To compute the activation temperature, one may directly use Ea/Ru, but unit consistency must be maintained.

1.5.5. OpenFOAM Implementation Details

1.5.5.1. Chemical Equilibrium Constant Limitation

In OpenFOAM's related computations of the reverse rate constant, to avoid numerical instability caused by an excessively small chemical equilibrium constant, a minimum-value limit is added to the denominator of the relevant division. The code for OpenFOAM-13 is shown below; to avoid a zero denominator, the max function is used to ensure that the denominator is not smaller than rootSmall.

```
template<class ThermoType, class ReactionRate>
Foam::scalar Foam::ReversibleReaction<ThermoType, ReactionRate>::kr
(
    const scalar kfwd,
    const scalar p,
    const scalar T,
    const scalarField& c,
    const label li
) const
{
    return kfwd/max(this->Kc(p, T), rootSmall);
}
...
template<class ThermoType, class ReactionRate>
Foam::scalar Foam::ReversibleReaction<ThermoType, ReactionRate>::dkrdT
(
    const scalar p,
    const scalar T,
    const scalarField& c,
    const label li,
    const scalar dkfdT,
    const scalar kr
) const
{
    const scalar Kc = max(this->Kc(p, T), rootSmall);
```

```
    return dkfdT/Kc - (Kc > rootSmall ? kr*this->dKcdTbyKc(p, T) : 0);
}
```

`rootSmall` is approximately 1.49e-8. This limitation has a rather serious flaw and leads to inaccuracies in the computation. Among all the zero-dimensional ignition delay time cases in this project, the Stagni ammonia reaction mechanism is relatively sensitive to this limitation; we used this mechanism to perform a zero-dimensional ignition problem solution test. The volume ratios of NH3, N2, and O2 are 4, 11.286, and 3, and the initial temperature and pressure are 1300 K and 3 MPa. The ODE solver is Rodas34, with absolute and relative tolerances of 10^{-12} and 10^{-6}. For OpenFOAM compatibility, **the PLOG reactions in the original Stagni mechanism were commented out and treated as standard Arrhenius reactions**. The Cantera setup refers to Constant-pressure, adiabatic kinetics simulation — Cantera 3.2.0 documentation

The table below provides three computational results for this mechanism: the temperatures at different times computed by OpenFOAM-10 with rootSmall retained, with rootSmall removed, and by Cantera-3.2. It can be found that after removing rootSmall, the OpenFOAM-10 results are almost identical to those of Cantera, whereas with rootSmall retained, the ignition delay time is noticeably shorter.

Result with rootSmall	Result without rootSmall	Result from Cantera	Relative error
[1399.99 K, 0.00096871 s]	[1399.99 K, 0.00166342 s]	[1400.01 K, 0.0016673 s]	41.76%
[1500.02 K, 0.00140371 s]	[1500.33 K, 0.00210342 s]	[1499.99 K, 0.0021078 s]	33.26%
[1599.90 K, 0.00155671 s]	[1600.23 K, 0.00225642 s]	[1599.96 K, 0.0022619 s]	31.01%
[1701.59 K, 0.00161667 s]	[1699.34 K, 0.00231541 s]	[1700.13 K, 0.0023216 s]	30.18%
[1799.26 K, 0.00164074 s]	[1799.79 K, 0.00234045 s]	[1799.73 K, 0.0023464 s]	29.90%
[1900.76 K, 0.00165199 s]	[1901.34 K, 0.00235167 s]	[1900.64 K, 0.0023576 s]	29.75%
[2000.02 K, 0.00165723 s]	[2000.30 K, 0.00235689 s]	[2000.94 K, 0.0023629 s]	29.68%
[2099.41 K, 0.00165996 s]	[2099.67 K, 0.00235962 s]	[2099.55 K, 0.0023656 s]	29.65%
[2199.05 K, 0.00166151 s]	[2199.33 K, 0.00236116 s]	[2203.99 K, 0.0023672 s]	29.63%
[2300.18 K 0.00166247 s]	[2300.49 K, 0.00236212 s]	[2298.97 K, 0.0023681 s]	29.62%
[2400.28 K, 0.00166313 s]	[2400.61 K, 0.00236277 s]	[2406.78 K, 0.0023688 s]	29.61%
[2499.46 K, 0.00166362 s]	[2499.81 K, 0.00236327 s]	[2506.78 K, 0.0023693 s]	29.61%
[2599.68 K, 0.00166406 s]	[2599.84 K, 0.00236371 s]	[2606.78 K, 0.0023698 s]	29.60%

Here, the reactions with Kc smaller than 1.49e-8 at initial state is **H2=H+H**, **NH3=NH2+H**, and **NH3+O2=NH2+HO2**

In OpenFOAM-14, this rootSmall limitation was removed, which is reasonable. Under the double-precision floating-point (IEEE 754) representation, for kf/Kc to cause numerical overflow (i.e., exceeding the representable range of 10^{308}), it would require $Kc < 10^{-308}$. If the equilibrium constant computed for some reaction within its applicable temperature range does reach this magnitude, it means that the reverse

contribution of that reaction can no longer be handled reliably in numerical terms; this usually suggests that the necessity of that reaction in the mechanism should be re-examined, rather than simply relying on numerical truncation to evade it.

1.5.5.2. JANAF Expression for the Chemical Equilibrium Constant

When computing the chemical equilibrium constant of a reversible reaction, it is usually necessary to compute the dimensionless standard Gibbs free energy of each species, and then compute the **change in Gibbs free energy after the reactants are completely converted into products**. To avoid redundant computation, `OpenFOAM-10` adopts a clever approach that fuses the `Gstd` function of each species. The computation code for its `Gstd` function is:

```
// Returns the mass-based standard Gibbs free energy, in units of J/kg
template<class EquationOfState>
inline Foam::scalar Foam::janafThermo<EquationOfState>::Gstd
(
    const scalar T
) const
{
    const coeffArray& a = coeffs(T);
    return
    (
        (
            a[0]*(1 - log(T))
          - (((a[4]/20.0*T + a[3]/12.0)*T + a[2]/6.0)*T + a[1]/2.0)*T
          - a[6]
        )*T
      + a[5]
    );
}
```

It should be noted that the array `a` provided here is not the `janaf` coefficients of a particular species, but the coefficients obtained by weighted-averaging all the species of the entire reaction as a whole; the total Gibbs free energy on the reactant side can be expressed by the following formula:

$$a_{\text{reaction}} = \frac{\sum_{\text{prod}} \nu_i^{''} W_i a_i}{\sum_{\text{prod}} \nu_i^{''} W_i} - \frac{\sum_{\text{react}} \nu_j^{'} W_j a_j}{\sum_{\text{react}} \nu_j^{'} W_j} \tag{73}$$

This a_{reaction} can now be directly substituted into the evaluation function $f(a, T)$ of the `JANAF` polynomial to obtain the change in Gibbs free energy per unit mass; however, what a chemical reaction requires is the molar-based change in standard Gibbs free energy. The final result must also be multiplied by $\sum_{\text{prod}} \nu_j W_j$, followed by exponentiation to obtain the partial-pressure-based chemical equilibrium constant. The code with which `OpenFOAM-10` computes the partial-pressure-based chemical equilibrium constant is:

```
template<class Thermo, template<class> class Type>
inline Foam::scalar
Foam::species::thermo<Thermo, Type>::K(const scalar p, const scalar T) const
{
    scalar arg = -this->Y()*this->Gstd(T)/(RR*T);
```

```
    if (arg < 600)
    {
        return exp(arg);
    }
    else
    {
        return rootVGreat;
    }

}

template<class Thermo, template<class> class Type>
inline Foam::scalar
Foam::species::thermo<Thermo, Type>::Kp(const scalar p, const scalar T) const
{
    return K(p, T);
}
```

Although the `OpenFOAM-10` approach is clever, it has a problem: the `janaf` polynomial has two sets of fitted coefficients for high and low temperatures, and the high/low-temperature ranges of each species may differ—some species may be 1000 K, others 1300 K. `OpenFOAM-10` uses the `Tcommon` of the left-hand species as the weighted-averaged `Tcommon` by default; for details, refer to the `+` and `+=` operator overloads in the janafThermoI.H file. This is clearly an implementation error.

1.5.5.3. Concentration Product Limitation

`OpenFOAM` performs a numerical-stability treatment on the concentration product to avoid abnormal values in the concentration product, as shown below; specifically, the concentration product is computed only when the species concentration is greater than `small`, or when the reaction order `el` and `er` of that species are greater than or equal to 1. This treatment is mainly aimed at global reactions; for elementary reactions, the reaction order is always greater than or equal to 1, so it is computed strictly according to the law of mass action.

```
template<class ReactionThermo>
void Foam::Reaction<ReactionThermo>::C
(
    const scalar p,
    const scalar T,
    const scalarField& c,
    const label li,
    scalar& Cf,
    scalar& Cr
) const
{
    Cf = Cr = 1;

    forAll(lhs(), i)
    {
        const label si = lhs()[i].index;
        const specieExponent& el = lhs()[i].exponent;
        Cf *= c[si] >= small || el >= 1 ? pow(max(c[si], 0), el) : 0;
    }
```

```
    forAll(rhs(), i)
    {
        const label si = rhs()[i].index;
        const specieExponent& er = rhs()[i].exponent;
        Cr *= c[si] >= small || er >= 1 ? pow(max(c[si], 0), er) : 0;
    }
}
```

1.5.5.4. Effect of Partial Derivatives and Their Approximations on Computational Efficiency

As mentioned earlier, the Jacobian matrix requires the partial derivatives of the reaction rate with respect to temperature and concentration. In theory, OpenFOAM-10 provides complete partial-derivative operations, but some of these computations are disabled by default; for example, the analytical partial derivative of the Falloff reaction rate constant with respect to temperature is as follows:

$$\frac{\partial k_f}{\partial T} = \frac{\partial k_\infty}{\partial T}\frac{Pr}{1+Pr}F + k_\infty \frac{1}{(1+Pr)^2}\frac{\partial Pr}{\partial T}F + k_\infty \frac{Pr}{1+Pr}\frac{\partial F}{\partial T} \tag{74}$$

To save computational cost, OpenFOAM adopts the following approximate formula by default:

$$\frac{\partial k_f}{\partial T} \approx \frac{\partial k_\infty}{\partial T}\frac{Pr}{1+Pr}F \tag{75}$$

See the following code block:

```
template<class ReactionRate, class FallOffFunction>
inline Foam::scalar
Foam::FallOffReactionRate<ReactionRate, FallOffFunction>::ddT
(
    const scalar p,
    const scalar T,
    const scalarField& c,
    const label li
) const
{
    const scalar k0 = k0_(p, T, c, li);
    const scalar kInf = kInf_(p, T, c, li);
    const scalar M = thirdBodyEfficiencies_.M(c);
    const scalar Pr = k0/kInf*M;
    const scalar F = F_(T, Pr);

    const scalar dkInfdT = kInf_.ddT(p, T, c, li);

    #if FALL_OFF_FUNCTION_JACOBIAN

    const scalar dk0dT = k0_.ddT(p, T, c, li);
    const scalar dPrdT = (M*dk0dT - Pr*dkInfdT)/kInf;
    const scalar dFdT = F_.ddT(T, Pr, F) + F_.ddPr(T, Pr, F)*dPrdT;

    return
```

```
            dkInfdT*(Pr/(1 + Pr))*F
          + kInf*dPrdT/sqr(1 + Pr)*F
          + kInf*(Pr/(1 + Pr))*dFdT;

    #else

    return dkInfdT*(Pr/(1 + Pr))*F;

    #endif
}
```

Similarly, the partial derivative of the rate constant with respect to species concentration also has exact and approximate expressions; the exact expression is:

$$\frac{\partial k_f}{\partial C_j} = \frac{\partial k_\infty}{\partial C_j}\frac{Pr}{1+Pr}F + k_\infty \frac{1}{(1+Pr)^2}\frac{\partial Pr}{\partial C_j}F + k_\infty \frac{Pr}{1+Pr}\frac{\partial F}{\partial Pr}\frac{\partial Pr}{\partial C_j} \tag{76}$$

The approximate expression adopted by OpenFOAM by default is:

$$\frac{\partial k_f}{\partial C_j} \approx \frac{\partial k_\infty}{\partial C_j}\frac{Pr}{1+Pr}F \tag{77}$$

ChemicallyActivated reactions also have a similar conditional macro to determine whether the exact or approximate expression is used, which needs to be modified in the ChemicallyActivatedReactionRateI.H file.

The table below lists the number of ODE time steps required when using approximate and exact partial derivatives in the zero-dimensional ignition test. It can be seen that when strict convergence tolerances are used, the difference in the required number of ODE time steps between the approximate and exact partial derivatives is very significant. When loose convergence tolerances are used, the difference in ODE time steps decreases. Of course, it also depends on the number and importance of these reactions among the total reactions. For example, for the NC7H16_32s_104r case, its reaction mechanism does not contain Falloff or ChemicallyActivated reactions, so the distinction between exact and approximate partial derivatives has no effect on this mechanism (the ODE step counts for the two are identical in the table).

	A10R5	**A10R5**	**A8R2**	**A8R2**
	Exact	**Approx**	**Exact**	**Approx**
C2H4_59s_271r	457	9117	80	124
C2H4_25s70r	398	551	49	49
C2H5OH_57s_383r	469	15056	74	138
C2H5OH_39s_370r	487	14011	76	154
C3H8_70s_463r	906	13701	49	140
CH3OH_116s_962r	372	15344	59	163
CH4_21s_84r	1260	12561	1037	1079
CH4_28s_116r	1221	8756	1035	1080
CH4_38s_291r	1454	16813	1220	1313
CH4_53s_325r	1260	18711	1039	1098

	A10R5	A10R5	A8R2	A8R2
CH4_68s_631r	1269	14621	1036	1103
CH4_59s_271r	1300	14070	1041	1110
DME_26s_71r	599	14396	166	267
DME_79s_351r	620	16552	172	249
DME_39s_175r	631	8408	174	230
H2_13s_27r	171	13268	45	129
H2_10s_21r	170	7699	47	84
H2_15s_71r	205	8784	61	132
IC8H18_140s_643r	874	9668	465	505
Kerosene_28s_92r	5300	9027	5046	5051
Kerosene_32s_77r	2590	2835	2082	2081
NC12H26_31s_193r	1348	13248	1050	1091
NC12H26_54s_269r	1340	12873	1049	1109
NC7H16_46s_115r	474	7634	95	150
NC7H16_68s_283r	471	11947	99	159
NC7H16_32s_104r	714	714	361	361
NH3CH4_129s_1231r	1358	12071	1073	1142
NH3CH4_59s_356r	1322	11365	1056	1117
NH3CH4_42s_130r	1346	15409	1061	1141
NH3CH4_125s_1099r	1370	13986	1060	1135
NH3_38s_232r	815	14042	50	151
NH3_32s_213r	382	9215	69	125
NH3_31s_200r	576	10014	322	395
NH3_38s_263r	319	11372	64	136

1.5.5.5. Number of Independent Species

The partial derivatives obtained by choosing Ns species as independent variables versus Ns-1 species as independent variables differ; one example is the third-body efficiency factor. For the third-body efficiency factor $M = \sum_{i=1}^{Ns} a_i C_i$, the partial derivative of the third-body efficiency factor with respect to the molar concentration of the i-th species differs between using $Ns - 1$ and Ns independent species.

When using all Ns molar concentrations $C_1, C_2, \ldots, C_{Ns}$ as independent variables, the third-body term is defined as:

$$M = \sum_{i=1}^{N_s} a_i C_i \tag{78}$$

Its partial derivative with respect to the i-th species is:

$$\frac{\partial M}{\partial C_i} = a_i, \qquad i = 1, \dots, N_s \tag{79}$$

When using $Ns - 1$ independent variables (i.e., $C_1, \dots, C_{Ns-1}$, with the last constrained by the equation of state):

$$M = \alpha_{Ns} C_{\text{tot}} + \sum_{i=1}^{N_s - 1} (\alpha_i - \alpha_{Ns}) C_i, \qquad C_{\text{tot}} = \frac{P}{R_u T} \tag{80}$$

The partial derivative with respect to the independent variable C_i ($i = 1, \dots, Ns - 1$) is then:

$$\frac{\partial M}{\partial C_i} = \alpha_i - \alpha_{Ns} \tag{81}$$

For Ns-1 independent species, when the third-body efficiency factor of a certain species is $\alpha_i = 1$, the partial derivative is 0, and the coupling between that species and the third-body reaction term in the Jacobian matrix is eliminated, thereby significantly increasing the sparsity of the matrix and favoring improved solution efficiency.

1.6. Summary of OpenFOAM Implementation Details

1. **Evolution of State-Variable Selection**
   In the Foundation version of OpenFOAM, versions up to and including OpenFOAM-9 use molar concentration as the state variable, whereas OpenFOAM-10 and later switch to species mass fractions. The reason is that when molar concentration is used, the contribution of the volume-change term under the adiabatic constant-pressure combustion assumption in operator splitting cannot be eliminated, so the newer versions switch to the mass-fraction form. At present, all versions treat all species as independent variables rather than exploiting the conservation constraint to select only Ns−1 species, which causes the computed partial-derivative matrix to differ from the latter.
2. **Completeness and Bug Fixes of the Jacobian Matrix**
   Versions up to and including OpenFOAM-5.x neglected the partial derivatives of the temperature rate of change with respect to temperature and species, making the Jacobian matrix incomplete and affecting solution stability. OpenFOAM-6 through OpenFOAM-8 added the computation of the partial derivatives of the temperature rate of change with respect to temperature and species, but the implementation contained an error (an extra redundant dTdt/T term was added to the partial derivative of the temperature rate of change with respect to temperature); this error was fixed in OpenFOAM-9.
3. **Inconsistency in the Adiabatic Assumption**
   In OpenFOAM-9 and earlier versions, the expression for the molar concentration rate of change is based on the adiabatic constant-volume assumption, whereas the temperature rate of change is based on the adiabatic constant-pressure assumption. The two assumptions are not completely consistent physically, which may have some impact on computational accuracy. In OpenFOAM-10, both the species mass-fraction rate of change and the temperature rate of change are based on the adiabatic constant-pressure assumption.
4. **Error in Handling the Volume-Change Term and Its Fix**
   OpenFOAM-10 erroneously used the new density when computing the mean reaction source term, causing the reaction source term to incorrectly include the contribution of the volume-change term. This issue was fixed in OpenFOAM-11.
5. **Version Differences of the Seulex Solver**

In the Foundation version, the Seulex solver adopted in OpenFOAM-5.x, OpenFOAM-14, and other versions exhibits implementation differences. The zero-dimensional ignition tests in this document show that OpenFOAM-14 has the fewest iterations and the best performance.

6. **Jacobian Matrix Approximation**
   OpenFOAM-10 theoretically provides a fully analytical Jacobian matrix, but to save computational cost it neglects by default the off-diagonal entries of the partial-derivative matrix of molar concentration with respect to species mass fraction, and also neglects some partial derivatives of pressure-dependent reactions. The test results in this document indicate that this may lead to an increase in iteration count and a decrease in accuracy.
7. **Stability Limitation**
   To avoid an infinitely large reverse reaction rate constant Kr when the equilibrium constant Kc=0, versions up to and including OpenFOAM-13 added the `rootSmall` variable to the denominator as a stability limit; however, this value was set too large, causing erroneous ignition delay time predictions (see the relevant section in this document). This issue was fixed in OpenFOAM-14.

After inspection, the reaction kinetics model of OpenFOAM-14 is likely the most accurate Foundation version to date; if an official solver is used, OpenFOAM-14 is recommended.

2. Computational Performance Optimization Strategies

2.1. Mathematical Expression Optimization

This subsection introduces mathematical-expression optimizations at the level of the reaction kinetics model and thermophysical model. Many mathematical expressions are theoretically equivalent yet differ substantially in the number of multiplications and additions they require; choosing an appropriate expression can directly and effectively improve the computational speed.

2.1.1. Rate Constant Evaluation

The computational expression for the forward rate constant is as follows:

$$k_f(T) = A\,T^{\beta}\exp\left(-\frac{E_a}{RuT}\right) \tag{82}$$

However, the following expression is superior in computational speed:

$$k_f(T) = A\,\exp\left(\beta\log T - \frac{E_a}{RuT}\right) \tag{83}$$

The reason is that we avoid the costly exponentiation operation, and $\log T$ does not need to be recomputed for every reaction—it is evaluated once at the beginning and then stored.

The evaluation of the reverse rate constant is slightly more involved. Reconsidering the expression for the partial-pressure equilibrium constant:

$$K_p = \exp\left(-\frac{\Delta G_{std}}{RuT}\right) \tag{84}$$

The term in parentheses denotes the dimensionless change in standard Gibbs free energy after fully converting reactants into products, given by:

$$-\frac{\Delta G_{std}}{RuT} = \frac{\sum_{k=1}^{N_s} v_k G_{k,std}}{RuT} \tag{85}$$

With the current expression, we need one exponential evaluation per reversible reaction to obtain the partial-pressure equilibrium constant. Following the suggestion of Law and Lu [2], we can transform it into the following equivalent form:

$$K_p = \prod_{i=1}^{N_s} \left[\exp\left(-\frac{G_{i,std}}{RuT}\right)\right]^{v_i} = \prod_{i=1}^{N_s} \left[\exp\left(B_i\right)\right]^{v_i} \tag{86}$$

Clearly, the new equivalent expression reduces the number of exponential evaluations from the number of reversible reactions to the number of species. If the number of reversible reactions exceeds the number of species, this modification is worthwhile.

2.1.2. Choice of State Variables

Since OpenFOAM-10, to accurately account for volume-change effects, the state variables were changed from molar concentrations to species mass fractions. However, choosing mole numbers as state variables can also accommodate volume changes while effectively reducing the operation count, especially in the Jacobian matrix computation. We now analyze the computational cost of the four sub-blocks of the Jacobian matrix in turn.

1. Matrix of partial derivatives of the species rate of change w.r.t. species

Let us first examine the matrix when species mass fraction is used as the state variable:

$$\begin{cases} \dfrac{\partial}{\partial Y_j}\left(\dfrac{dY_i}{dt}\right) = \dfrac{W_i}{\rho}\displaystyle\sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_i}{\partial C_k}\frac{\partial C_k}{\partial Y_j} - \frac{dY_i}{dt}\frac{1}{\rho}\frac{\partial \rho}{\partial Y_j} \\ \dfrac{\partial C_i}{\partial Y_j} = \dfrac{\partial \rho}{\partial Y_j}\dfrac{Y_i}{W_i} + \dfrac{\rho}{W_i}\delta_{ij} \end{cases} \tag{87}$$

The expression above is the sub-matrix of the Jacobian containing the partial derivatives of the species mass-fraction rate of change w.r.t. the species mass fractions. We first need to construct the partial-derivative matrix of molar concentration w.r.t. species mass fraction, $\frac{\partial C_i}{\partial Y_j}$. Assuming the arrays on the right-hand side of $\frac{\partial C_i}{\partial Y_j}$ are all known, evaluating $\frac{\partial C_i}{\partial Y_j}$ requires approximately $(Ns^2 + Ns)$ multiplications and (Ns) additions, giving an overall time complexity of $O(Ns^2)$—that is, the theoretical computation time scales quadratically with the number of species. Evaluating the matrix $\frac{\partial}{\partial Y_j}\left(\frac{dY_i}{dt}\right)$ requires $(Ns^3 + 3Ns^2)$ multiplications and $\left(Ns^3\right)$ additions, with an overall time complexity of $O(Ns^3)$, i.e., the computation time scales cubically with the number of species, and the workload grows rapidly as the species count increases. This is why OpenFOAM-10 provides an approximation option that reduces the complexity to $O(Ns^2)$.

Next, consider the partial-derivative matrix when the mole number is used as the state variable. At first glance, this formula appears to have a time complexity of $O(Ns^3)$. However, the summation term $S_i = \sum_{k=1}^{N_s}\frac{\partial \dot{\omega}_i}{\partial C_k}C_k$ needs to be evaluated only once at the start of each row and can then be reused along the column. Thus the actual number of multiplications is $Ns^2 + Ns$, and the number of additions is $2Ns^2$, yielding an overall theoretical time complexity of $O(Ns^2)$.

$$\frac{\partial}{\partial N_j}\left(\frac{dN_i}{dt}\right) = \left(\dot{\omega}_i - \sum_{k=1}^{N_s} \frac{\partial \dot{\omega}_i}{\partial C_k} C_k\right) \frac{V}{N} + \frac{\partial \dot{\omega}_i}{\partial C_j} \tag{88}$$

Given the time-complexity advantage of mole-number variables in the Jacobian computation, this solver additionally provides a computation path based on mole numbers as state variables, which the user can switch to at runtime depending on the mechanism size.

2.2. Code Optimization

2.2.1. Instruction-Level Parallelism

The circuits within a CPU can be roughly divided into combinational circuits and sequential circuits. A **combinational circuit** produces an output that depends **only on the current inputs** and is independent of past states (e.g., the arithmetic logic unit and the decoder). A **sequential circuit**, by contrast, has outputs that also depend on the circuit's own state, which in turn depends on previous inputs (analogous to a recurrence relation in mathematics); sequential circuits include devices such as counters and registers.

To improve CPU execution efficiency, the most direct method is to raise the clock frequency using a crystal oscillator and a phase-locked loop, so that the CPU performs more voltage transitions per second and naturally processes faster. This approach, however, is limited by many factors, one of which is the physical time required to execute an instruction. For example, to execute a floating-point computation instruction, the CPU roughly follows these steps:

1. Instruction fetch: retrieve the next instruction from the instruction memory.
2. Decode: the CPU obtains the specific content of the instruction, e.g., which execution unit to use and which memory location to read.
3. Read: load data from memory or cache into registers.
4. Execute: perform the floating-point computation via the floating-point unit.
5. Write: store the result back to memory.

These steps involve changes in semiconductor voltages, variations in memory capacitance, and the propagation of electromagnetic fields along the circuit—each step requires real physical time. If the clock frequency is forced higher, the consequence is that before the data has been read into the register file, the floating-point unit receives a new rising-voltage signal and attempts to fetch data from the register file, ultimately causing system failure.

Due to power-consumption and thermal-dissipation limits, relying solely on higher clock frequencies for performance gains has become increasingly difficult. Modern CPUs therefore rely more on **architectural optimization**, such as pipelining. In early single-instruction multi-cycle CPUs, the next instruction could only be fetched after the current one had fully retired; modern CPUs execute in a pipelined manner, so that once an instruction enters the decode stage, the next instruction can enter the fetch stage. Although pipelining does not reduce instruction latency, it raises instruction **throughput**: where previously one instruction might execute per minute, a 5-stage pipeline can execute five instructions per minute.

To further increase instruction execution rate, modern CPUs also employ the following techniques:

1. **Out-of-order execution**: the CPU need not execute instructions strictly in the order of the assembly or machine code; instead, it executes them according to the principle of maximizing CPU resource utilization. Note, however, that instruction fetch and decode are usually performed in order.

2. **Superscalar**: allows multiple instances of the same type of instruction to be executed simultaneously—for example, the front-end fetch unit and parallel decoders can fetch and decode several instructions at once. The back end is typically equipped with multiple execution units to run instructions in parallel; modern CPUs, for instance, usually have at least two floating-point multiplication or addition units.
3. **Branch prediction**: when the CPU encounters a conditional branch, it attempts to predict the branch and executes instructions along the predicted path, in order to avoid pipeline stalls. If the prediction is wrong, however, all speculatively executed instructions must be discarded, incurring a performance penalty.

These techniques are not panaceas, though: if instructions exhibit data hazards or complex conditional branches, the pipeline may stall. The lesson for us is to eliminate data hazards and conditional branches as much as possible.

To illustrate this, consider an example that appears strongly dependent. For an elementary reaction such as A+B=C+D, we wrote C++ code that updates the species production rates, shown below. Here `q` is the rate of the elementary reaction, and `omega_sA_increment` is the increment to the production rate of species A contributed by this reaction. `dNdtByV` holds the array of species production rates, and `sA` is the index of species A. The code below shows two ways of writing it.

```
void updateOmega(double* __restrict__ dNdtByV,double q,int sA,int sB,int sC, int sD)
{
    // First approach
    const double omega_sA_increment = dNdtByV[sA] + q;
    const double omega_sB_increment = dNdtByV[sB] + q;
    const double omega_sC_increment = dNdtByV[sC] - q;
    const double omega_sD_increment = dNdtByV[sD] - q;
    dNdtByV[sA] = omega_sA_increment;
    dNdtByV[sB] = omega_sB_increment;
    dNdtByV[sC] = omega_sC_increment;
    dNdtByV[sD] = omega_sD_increment;
    // Second approach
    //dNdtByV[sA] = dNdtByV[sA] + q;
    //dNdtByV[sB] = dNdtByV[sB] + q;
    //dNdtByV[sC] = dNdtByV[sC] - q;
    //dNdtByV[sD] = dNdtByV[sD] - q;
}
```

For the first approach, the assembly instructions are as follows. This assembly invokes four floating-point registers simultaneously for the arithmetic operations. As can be seen from the assembly, the four groups of floating-point-register-related instructions have weak dependencies and can be executed in an "**out-of-order** + **superscalar**" manner.

```
        movslq  %ecx, %rcx          ; store sC into %rcx
        movslq  %r8d, %r8           ; store sD into %r8
        movslq  %esi, %rsi          ; store sA into %rsi
        movslq  %edx, %rdx          ; store sB into %rdx

        vmovsd  (%rdi,%rcx,8), %xmm1 ; load dNdtByV[sC] into %xmm1
        leaq    (%rdi,%rsi,8), %r9   ; compute address of dNdtByV[sA], store in %r9 (for
later store)
```

```
        leaq    (%rdi,%rdx,8), %rsi  ; compute address of dNdtByV[sB], store in %rsi (for
later store)

        vaddsd  (%rsi), %xmm0, %xmm3 ; compute dNdtByV[sB] + q, store result temporarily in
%xmm3
        vsubsd  %xmm0, %xmm1, %xmm2 ; compute dNdtByV[sC] - q, store result temporarily in
%xmm2 (sC is a product, generated)
        vmovsd  (%rdi,%r8,8), %xmm1 ; load dNdtByV[sD] into %xmm1
        vsubsd  %xmm0, %xmm1, %xmm1 ; compute dNdtByV[sD] - q, store result temporarily in
%xmm1 (sD is a product, generated)
        vaddsd  (%r9), %xmm0, %xmm0 ; compute dNdtByV[sA] + q, store result temporarily in
%xmm0 (sA is a reactant, consumed)

        vmovsd  %xmm0, (%r9)         ; write result back to dNdtByV[sA]
        vmovsd  %xmm3, (%rsi)        ; write result back to dNdtByV[sB]
        vmovsd  %xmm2, (%rdi,%rcx,8) ; write result back to dNdtByV[sC]
        vmovsd  %xmm1, (%rdi,%r8,8)  ; write result back to dNdtByV[sD]
```

Now consider the assembly of the second C++ writing style. We find that the compiler uses only the two registers `xmm0` and `xmm1`. Why? Because the compiler does not know whether the species indices `sA`, `sB`, `sC`, `sD` are identical. If any index were the same, the first approach would produce an error (here, for the elementary reaction A+B=C+D, the indices differ). The compiler therefore prefers to emit conservative assembly, as shown below, where in principle dependencies exist among the instructions.

```
        movslq  %esi, %rsi          ; sign-extend sA (32-bit) to 64-bit, store in %rsi
        movslq  %edx, %rdx          ; sign-extend sB (32-bit) to 64-bit, store in %rdx
        movslq  %ecx, %rcx          ; sign-extend sC (32-bit) to 64-bit, store in %rcx
        movslq  %r8d, %r8           ; sign-extend sD (32-bit) to 64-bit, store in %r8

        vaddsd  (%rdi,%rsi,8), %xmm0, %xmm1 ; load dNdtByV[sA], compute dNdtByV[sA] + q
        vmovsd  %xmm1, (%rdi,%rsi,8)        ; write result back to dNdtByV[sA]

        vaddsd  (%rdi,%rdx,8), %xmm0, %xmm1 ; load dNdtByV[sB], compute dNdtByV[sB] + q
        vmovsd  %xmm1, (%rdi,%rdx,8)        ; write result back to dNdtByV[sB]

        vmovsd  (%rdi,%rcx,8), %xmm1        ; load dNdtByV[sC]
        vsubsd  %xmm0, %xmm1, %xmm1         ; compute dNdtByV[sC] - q
        vmovsd  %xmm1, (%rdi,%rcx,8)        ; write result back to dNdtByV[sC]

        vmovsd  (%rdi,%r8,8), %xmm1         ; load dNdtByV[sD]
        vsubsd  %xmm0, %xmm1, %xmm1         ; compute dNdtByV[sD] - q
        vmovsd  %xmm1, (%rdi,%r8,8)         ; write result back to dNdtByV[sD]
```

In reality, however, the CPU does not literally use only `xmm0` and `xmm1`. Although the x86 instruction set exposes only 16 floating-point registers to assembly, the CPU may internally have hundreds of floating-point registers; while executing the above instructions, the CPU will use idle floating-point registers rather than truly reusing the same one.

Below is an example with a genuine dependency—two code implementations for computing the third-body efficiency—shown in C++ as follows:

```
// Version 1
double M
(
    int n,
    double* __restrict__ a,
    double* __restrict__ c
)
{
    double m = 0;
    for(int i =0;i<n;i++)
    {
        m = m + a[i]*c[i];
    }
    return m;
}
// Version 2
double M2
(
    int n,
    double* __restrict__ a,
    double* __restrict__ c
)
{
    int remain = n%2;
    double m0 = 0;
    double m1 = 0;

    for(int i =0;i<n-remain;i=i+2)
    {
        m0 = m0 + a[i+0]*c[i+0];
        m1 = m1 + a[i+1]*c[i+1];
    }
    if(remain==1)
    {
        int i = n-remain;
        m0 = m0 + a[i+0]*c[i+0];
    }
    return m0+m1;
}
```

The **main loop** portion of the assembly is shown below. To focus solely on instruction-level parallelism and avoid the complexity introduced by compiler auto-vectorization, the optimization level was set to `-O1`. It is easy to see that function `M` has only one data-dependency chain, whereas function `M2` has two independent data-dependency chains. The CPU can execute the instructions on the two chains of `M2` in parallel.

```
M(int, double*, double*):
#...
.L3:
        movsd   (%rsi,%rax), %xmm0
        mulsd   (%rdx,%rax), %xmm0
        addsd   %xmm0, %xmm1
        addq    $8, %rax
```

```
        cmpq    %rcx, %rax
        jne     .L3
#...

M2(int, double*, double*):
.L8:
        movsd   (%rsi,%rax,8), %xmm1
        mulsd   (%rdx,%rax,8), %xmm1
        addsd   %xmm1, %xmm0
        movsd   8(%rsi,%rax,8), %xmm1
        mulsd   8(%rdx,%rax,8), %xmm1
        addsd   %xmm1, %xmm2
        addq    $2, %rax
        cmpl    %eax, %edi
        jg      .L8
```

2.2.2. SIMD

Next, the SIMD technique is introduced. Modern CPU instruction sets typically extend SIMD (Single Instruction Multiple Data) support. SIMD allows a single CPU instruction to operate on multiple data elements simultaneously. For example, the AVX2 (Advanced Vector Extensions) instruction set under the x86 architecture can use 256-bit-wide registers to load four double-precision floating-point numbers (64 bits each) at once, then complete the addition of these four numbers with a single vector-add instruction. Without vectorization, the compiler would generate four independent add instructions, adding them pair by pair.

Compilers such as GCC and Clang are capable of automatic vectorization. At compile time, when the compiler recognizes that there is no loop-carried data dependency in a loop and that every iteration performs an identical operation, it automatically replaces scalar instructions with vector instructions. The success of auto-vectorization, however, depends heavily on how the code is written. The following code patterns prevent the compiler from vectorizing:

1. **Loop-carried data dependency**: the computation of a later iteration depends on the result of an earlier iteration. For example, a reduction such as `sum = sum + a[i]` can be implemented with special vector-reduction instructions, but requires explicitly enabling or adjusting the optimization level.
2. **Non-contiguous memory access pattern**: vectorization requires the loaded data to be stored contiguously in memory (i.e., unit stride). When accessing a 2-D array column-wise or accessing array elements with irregular strides, the compiler cannot generate efficient vector-load instructions.
3. **Complex branch conditions within the loop**: when the loop body contains `if-else` constructs and different iterations take different branches, the efficiency after vectorization drops sharply, because all elements in the vector register must execute the code of both branches. Alternatively, overly complex branch conditions may cause the compiler to give up vectorization.
4. **Pointer aliasing risk**: if the compiler cannot determine whether two pointers point to the same memory address, it will conservatively generate scalar code to avoid potential data-overwrite issues. In OpenFOAM source code, this problem can be mitigated via the `__restrict__` keyword or the compiler's aliasing-analysis options.

A C++ example for verifying vectorization is presented below; it evaluates the forward rate constant of elementary reactions.

```
void Kf
(
    double* A,
    double* Ta,
    double* beta,
    double* Kf,
    double T,
    int n
)
{
    for(int j=0;j<n;j++)
    {
        Kf[j] = A[j]*std::pow(T,beta[j])*std::exp(Ta[j]/T);
    }
}
```

At compile time, to enable vectorization, the `-O3 -ffast-math` options must be added for GCC; otherwise GCC will conservatively implement scalar arithmetic. With these options, the assembly GCC generates first checks, based on the variable `n` and the pointers' base addresses, whether aliasing exists. If so, it falls back to a pure scalar loop; otherwise it invokes vector operations. The main portion of the vector-operation assembly is as follows.

```
        movupd  0(%rbp,%r14), %xmm2
        movupd  (%r12,%r14), %xmm1
        movapd  32(%rsp), %xmm0
        movaps  %xmm2, (%rsp)
        call    _ZGVbN2vv_pow
        movaps  %xmm0, 16(%rsp)
        movupd  0(%r13,%r14), %xmm0
        mulpd   48(%rsp), %xmm0
        call    _ZGVbN2v_exp
        mulpd   16(%rsp), %xmm0
        mulpd   (%rsp), %xmm0
        movups  %xmm0, (%rbx,%r14)
        addq    $16, %r14
        cmpq    %r14, %r15
        jne     .L6
        movl    92(%rsp), %eax
        testb   $1, %al
        je      .L3
        movl    %eax, %r14d
        movsd   80(%rsp), %xmm0
        andl    $-2, %r14d
        movsd   (%r12,%r14,8), %xmm1
        call    pow
        movsd   72(%rsp), %xmm6
        mulsd   0(%r13,%r14,8), %xmm6
        movsd   %xmm0, (%rsp)
        movapd  %xmm6, %xmm0
        call    exp
        mulsd   (%rsp), %xmm0
        mulsd   0(%rbp,%r14,8), %xmm0
```

```
        movsd   %xmm0, (%rbx,%r14,8)
```

Note the instructions "`call _ZGVbN2vv_pow`" and "`call _ZGVbN2v_exp`" in the assembly above—these invoke GCC's vectorized power and exponential functions, each processing two elements at a time. Subsequently, if the variable `n` is not a multiple of 2, we see a single `pow` and a single `exp` call below to handle the tail elements.

As mentioned earlier, if aliasing exists, scalar instructions complete the whole loop, as seen in the `.L8` code segment below, which uses `movsd`, `mulsd`, and the `pow` and `exp` functions. Under normal circumstances, rate-constant evaluation should have no aliasing, so adding the `__restrict__` keyword to the four pointer parameters makes the `.L8` segment disappear.

```
.L8:
        movsd   (%r12,%r15), %xmm1
        movsd   80(%rsp), %xmm0
        call    pow
        movsd   72(%rsp), %xmm5
        mulsd   0(%r13,%r15), %xmm5
        movsd   %xmm0, (%rsp)
        movapd  %xmm5, %xmm0
        call    exp
        mulsd   (%rsp), %xmm0
        mulsd   0(%rbp,%r15), %xmm0
        movsd   %xmm0, (%rbx,%r15)
        addq    $8, %r15
        cmpq    %r14, %r15
        jne     .L8
```

If the CPU supports the AVX2 instruction set, the `-mavx2` option can be added at compile time, which generates 256-bit assembly instructions that process four 64-bit floating-point numbers at once. Note that the vectorized math functions become `_ZGVdN4vv_pow` and `_ZGVdN4v_exp`, and the register names change from `xmm` to `ymm`.

```
.L7:
        vmovupd (%r12,%r15), %ymm2
        vmovupd 0(%r13,%r15), %ymm1
        vmovapd -144(%rbp), %ymm0
        vmovapd %ymm2, -80(%rbp)
        call    _ZGVdN4vv_pow
        vmovapd -176(%rbp), %ymm3
        vmovapd %ymm0, -112(%rbp)
        vmulpd  (%r14,%r15), %ymm3, %ymm0
        call    _ZGVdN4v_exp
        vmulpd  -112(%rbp), %ymm0, %ymm0
        vmulpd  -80(%rbp), %ymm0, %ymm0
        vmovupd %ymm0, (%rbx,%r15)
        addq    $32, %r15
        cmpq    -184(%rbp), %r15
        jne     .L7
```

If the CPU supports the AVX512 instruction set, adding `-mavx512f` at compile time generates 512-bit floating-point operations that compute eight 64-bit floating-point numbers at once, as follows:

```
.L7:
        vmovupd (%r12,%r15), %zmm2
        vmovupd 0(%r13,%r15), %zmm1
        vmovapd -304(%rbp), %zmm0
        vmovapd %zmm2, -112(%rbp)
        call    _ZGVeN8vv_pow
        vmovapd -368(%rbp), %zmm3
        vmovapd %zmm0, -176(%rbp)
        vmulpd  (%r14,%r15), %zmm3, %zmm0
        call    _ZGVeN8v_exp
        vmulpd  -176(%rbp), %zmm0, %zmm0
        vmulpd  -112(%rbp), %zmm0, %zmm0
        vmovupd %zmm0, (%rbx,%r15)
        addq    $64, %r15
        cmpq    %r15, -184(%rbp)
        jne     .L7
```

Some C++ code written for a specific reaction mechanism uses hard-coding, with the reaction-kinetics parameters written as compile-time constants. When attempting vectorized computation in such cases, one must be careful that the compiler applies other optimizations that affect the implementation. For example, the two code blocks below produce different assembly, both compiled with `-O3 -ffast-math -mavx2`; block 1 is successfully vectorized while block 2 is not. The reason is that `pow(T, 1.0)` is constant-folded at compile time into `T`, degrading the operation to a scalar multiplication. Because a scalar operation is mixed into the basic block, the compiler cannot emit a uniform vector instruction for the entire basic block and therefore abandons vectorization in favor of fully scalar execution.

```
// Code block 1
    Kf[0] = 0.1*std::pow(T,0.01)*std::exp(0.01/T);
    Kf[1] = 1.1*std::pow(T,0.11)*std::exp(1.11/T);
    Kf[2] = 2.1*std::pow(T,0.21)*std::exp(2.22/T);
    Kf[3] = 3.1*std::pow(T,0.31)*std::exp(3.33/T);
// Code block 2
    Kf[0] = 0.1*std::pow(T,1.0)*std::exp(0.01/T);
    Kf[1] = 1.1*std::pow(T,0.11)*std::exp(1.11/T);
    Kf[2] = 2.1*std::pow(T,0.21)*std::exp(2.22/T);
    Kf[3] = 3.1*std::pow(T,0.31)*std::exp(3.33/T);
```

If vectorized computation is required, a better approach is to write intrinsic vectorization instructions directly, as shown below; invoking the vector instructions explicitly avoids the problem of unsuccessful compiler vectorization. In the code, `vec256_powd` and `vec256_expd` are vectorized math functions that must be provided by the math library.

```
__m256d Tv = _mm256_set1_pd(T);
__m256d Av = _mm256_setr_pd(0.1,1.1,2.1,3.1);
__m256d betav = _mm256_setr_pd(1.0,0.11,0.21,0.31);
__m256d Tav = _mm256_setr_pd(0.01,1.11,2.22,3.33);
__m256d powv = vec256_powd(Tv,betav);
__m256d expv = vec256_expd(_mm256_div_pd(Tav,Tv));
__m256d Kfv = _mm256_mul_pd(powv,expv);
Kfv = _mm256_mul_pd(Kfv,Av);
```

2.2.3. Memory Access

Compared with floating-point computation, memory access is much slower: reading data from main memory into registers typically takes hundreds of clock cycles, whereas reading from cache is far faster—as little as a few clock cycles. Therefore, improving cache hit rate is one of the key goals of program performance optimization.

The design of the memory hierarchy relies mainly on two locality principles: spatial locality and temporal locality. Spatial locality means that if a program accesses some data, nearby data are also likely to be accessed soon. Hence, when the CPU reads one datum, it often loads adjacent data into the cache together. Temporal locality means that accessed data are likely to be used again shortly—this is also why the CPU temporarily stores data in the cache; when cache space is insufficient, the least-frequently accessed data are evicted.

In chemical-reaction computation, the part most closely related to memory access is usually matrix processing—for example, building the Jacobian matrix, performing LU decomposition, and the solve process. For a row-major stored matrix, accessing data column-wise causes strided address access, easily triggering cache misses and significantly degrading efficiency.

2.2.4. Thread-Level Parallelism

This project does not consider thread-level parallelism for the time being. In fact, once the pipeline utilization has been pushed to a high level through the optimizations above, intra-core multi-threaded concurrency would interfere with itself due to contention for shared resources (cache/execution units). If OS-level thread switching is involved, it would further flush the pipeline and incur context-switch overhead, actually lowering efficiency. The main value of threading lies in handling I/O waits or improving interactive responsiveness (e.g., an e-commerce system interleaving order transactions concurrently to ensure each request receives timely feedback), rather than strengthening the peak compute power of purely computational tasks.

2.2.5. Process-Level Parallelism

In 3-D combustion, different sub-regions may have different thermophysical states—some regions may be burning while others are not. In burning regions, the ODE system needs more sub-time steps; in non-burning regions it needs fewer. This causes load imbalance among different processes.

This solver has developed a load-balancing module based on OpenFOAM's parallel library. The idea is to first compute the average load, then send high-load mesh cells to low-load cells for computation, and send the results back to the low-load cells after completion. This approach differs somewhat from OpenFOAM-10's implementation, which re-partitions the mesh cells.

Nevertheless, readers are still encouraged to try OpenFOAM-10's load-balancing method: the mesh re-partitioning approach easily accommodates multi-physics, for example where some processes hold a large number of spray particles and ISAT can be enabled for acceleration.

2.3. Optimization Examples

2.3.1. Forward Rate Constant

In chemical-reaction source-term computation, evaluation of the forward rate constant Kf is one of the most frequently called kernel functions. The `Arrhenius`-form expression for the forward rate constant is:

$$k_f(T) = A\,T^{\beta}\exp\left(-\frac{E_a}{RuT}\right) \tag{89}$$

In OpenFOAM's original implementation, each chemical reaction is encapsulated as an independent `ArrheniusReactionRate` object, each with its own `operator()` member function, invoked one by one through the virtual-function mechanism. Taking OpenFOAM-10 as an example, its core code is shown below:

```
inline Foam::scalar Foam::ArrheniusReactionRate::operator()
(
    const scalar p,
    const scalar T,
    const scalarField&,
    const label
) const
{
    scalar ak = A_;
    if (mag(beta_) > vSmall) { ak *= pow(T, beta_); }
    if (mag(Ta_) > vSmall) { ak *= exp(-Ta_/T); }
    return ak;
}
```

Although this object-oriented design offers good code maintainability and extensibility, it has efficiency bottlenecks at the following three levels in performance-sensitive computation scenarios:

1. **Virtual-function call overhead**: each reaction object calls the rate function indirectly through the virtual-function table; the accumulated overhead of many reactions is non-negligible, and the indirect call prevents the compiler from inlining, limiting further optimization headroom.
2. **Per-reaction serial processing**: all reactions are computed sequentially in container order. Even though reactions are fully independent, SIMD vectorization cannot be exploited because data such as A, beta, and Ta are not stored contiguously.

To address the above problems, optimization can be performed at the levels of mathematical-expression reorganization and instruction-level vectorization:

1. **Mathematical-expression optimization:** rewrite the Arrhenius formula into a unified exponential form, eliminating the `pow(T, beta_)` call in the original code.
2. **Explicit vectorization:** in this project, the forward-rate-constant arrays `A`, `beta`, `Ta`, and `Kf` are all stored as contiguous in-memory `double` arrays. We use the AVX2 instruction set (256-bit vector registers) to pack the forward-rate evaluation of every 4 reactions into one vectorization batch. The key logic of the implementation is shown below (only the core computation is shown; tail elements are handled by a scalar loop and are omitted from the snippet):
3. **Skip strategy for temperature-independent reactions:** for reactions with activation energy `Ta=0` and temperature exponent `β=0`, the forward rate constant degenerates to `kf=A`, i.e., it does not vary with

temperature. Such reactions (typically some unimolecular reactions at the low-pressure limit or specific stabilized reactions) account for a certain fraction of the mechanism; evaluating them through the full exponential-function path incurs unnecessary overhead. In this project, we count such reactions at initialization as `n_Temperature_Independent_Reaction`, and in the vectorization loop process only the remaining temperature-dependent reactions; the former are assigned directly without entering the loop.

```
__m256d LogT = _mm256_set1_pd(logT);
__m256d InvT = _mm256_set1_pd(-invT);
unsigned int times = (end - n_Temperature_Independent_Reaction) / 4;
for (unsigned int z = 0; z < times; z++) {
    unsigned int i = z * 4 + n_Temperature_Independent_Reaction;
    __m256d A_  = _mm256_loadu_pd(&A_ptr[i]);
    __m256d beta_ = _mm256_loadu_pd(&beta_ptr[i]);
    __m256d Ta_  = _mm256_loadu_pd(&Ta_ptr[i]);
    __m256d Kf = _mm256_fmadd_pd(beta_, LogT, _mm256_mul_pd(Ta_, InvT));
    Kf = vec256_expd(Kf);
    Kf = _mm256_mul_pd(A_, Kf);
    _mm256_storeu_pd(&Kf_ptr[i], Kf);
}
```

2.3.2. Concentration Product

In chemical-reaction source-term computation, assembling the concentration-product term is one of the most frequent core operations in each reaction evaluation. OpenFOAM implements this via the `Reaction::C()` function, whose original code is shown below. Here `Cf` is the reactant concentration product and `Cr` is the product concentration product. `lhs()` and `rhs()` return the lists of reactants and products, respectively; each list contains the species index `si`, the reaction order `el` (for the reactant side) and `er` (for the product side). `pow` is the power operation; OpenFOAM provides a fast-power implementation for integer reaction orders, used when the order is an integer (involving shift operations). `small` is a small value, approximately 2.22×10^{-16} (double precision). The `max` function ensures a species molar concentration is not less than 0.

```
template<class ReactionThermo>
void Foam::Reaction<ReactionThermo>::C
(
    const scalar p,
    const scalar T,
    const scalarField& c,
    const label li,

    scalar& Cf,
    scalar& Cr
) const
{
    Cf = Cr = 1;
    forAll(lhs(), i) {
        const label si = lhs()[i].index;
        const specieExponent& el = lhs()[i].exponent;
        Cf *= c[si] >= small || el >= 1 ? pow(max(c[si], 0), el) : 0;
    }
```

```
    forAll(rhs(), i) {
        const label si = rhs()[i].index;
        const specieExponent& er = rhs()[i].exponent;
        Cr *= c[si] >= small || er >= 1 ? pow(max(c[si], 0), er) : 0;
    }
}
```

This implementation is highly general, yet it has the following performance bottlenecks:

1. **Loop overhead**: every reaction must iterate over its reactant and product lists (even though the vast majority of elementary reactions have only 1–2 reactants or products); the conditional branching and function-call overhead inside the loop far exceed the actual multiplication itself.
2. **Power-function calls**: even though the `pow` function internally selects a fast-power path when the order is an integer, its overhead is still several times that of a direct multiplication. For elementary reactions with order 1 or 2, concentration production can be completed with one or two multiplications.
3. **Stability limitation**: for elementary reactions, the ternary operator in OpenFOAM is redundant.

To address the above issues, this project classifies all reactions by reaction type during mechanism initialization and builds an independent index array for each type, thereby completely eliminating the `forAll` loop and the `pow` call in the source-term computation. For non-elementary reactions, OpenFOAM-10's form is still used. The concentration-product code for an elementary reaction with exactly two reactants and two products is shown below.

```
std::size_t endRR22 = RR22Size;
std::size_t remainRR22 = endRR22 % 4;
for (std::size_t k = 0; k < endRR22 - remainRR22; k += 4) {
    // retrieve index of the k-th reaction and the concentration-product operation
    const unsigned int i0  = this->RR22AllIndex[k*5 + 0];
    const unsigned int sl0a = this->RR22AllIndex[k*5 + 1];
    const unsigned int sl1a = this->RR22AllIndex[k*5 + 2];
    const unsigned int sr0a = this->RR22AllIndex[k*5 + 3];
    const unsigned int sr1a = this->RR22AllIndex[k*5 + 4];
    const double CF0 = c[sl0a] * c[sl1a];
    const double CR0 = c[sr0a] * c[sr1a];
    // ... continue processing the k+1 reaction
    const unsigned int i1  = this->RR22AllIndex[k*5 + 5];
    const unsigned int sl0b = this->RR22AllIndex[k*5 + 6];
    const unsigned int sl1b = this->RR22AllIndex[k*5 + 7];
    const unsigned int sr0b = this->RR22AllIndex[k*5 + 8];
    const unsigned int sr1b = this->RR22AllIndex[k*5 + 9];
    const double CF1 = c[sl0b]*c[sl1b];
    const double CR1 = c[sr0b]*c[sr1b];
    // continue processing the k+2 and k+3 reactions
}
```

The specific improvements are described as follows:

1. **Classification by reaction type**: all reactions are divided into nine elementary-reaction types (1-1, 1-2, 1-3, 2-1, 2-2, 2-3, 3-1, 3-2, 3-3) according to their numbers of reactant and product species, plus two

global-reaction types involving multiple species. Each type has its own independent index array, which stores reaction numbers and the corresponding species numbers contiguously. Taking the 2-2 type as an example, the index array `RR22AllIndex` stores groups of five integers consecutively: the global reaction number, the species numbers of the two reactants, and the species numbers of the two products.

2. **Direct assembly of concentration products:** after the classification above, the concentration-product expression for each reaction type is fixed at compile time. For elementary reactions, the `max` and `pow` functions are avoided.
3. **Loop unrolling to raise instruction parallelism:** manually unrolling the loop to 4 reactions per batch helps modern CPUs fully exploit the multiple floating-point units in a superscalar architecture for parallel execution. The computations among reactions within the loop body are completely independent with no data dependency, enabling the processor to execute multiple multiplication instructions simultaneously and thus raising the instruction-level parallelism. During the write-back stage, data dependencies may exist between reactions (write-after-write due to repeated species); this can be resolved by relying on CPU scheduling or by further splitting the reaction types.

2.3.3. Chemical Equilibrium Constant

For the reverse rate constant, OpenFOAM first computes the chemical equilibrium constant `Kc` via the thermophysical module, as shown below. Because one exponential evaluation is required per reversible reaction, this is inefficient.

```
template<class Thermo, template<class> class Type>
inline Foam::scalar
Foam::species::thermo<Thermo, Type>::K(const scalar p, const scalar T) const
{
    scalar arg = -this->Y()*this->Gstd(T)/(RR*T);

    if (arg < 600)
    {
        return exp(arg);
    }
    else
    {
        return rootVGreat;
    }
}

template<class Thermo, template<class> class Type>
inline Foam::scalar
Foam::species::thermo<Thermo, Type>::Kp(const scalar p, const scalar T) const
{
    return K(p, T);
}

template<class Thermo, template<class> class Type>
inline Foam::scalar
Foam::species::thermo<Thermo, Type>::Kc(const scalar p, const scalar T) const
```

```
{
    const scalar nm = this->Y()/this->W();

    if (equal(nm, small))
    {
        return Kp(p, T);
    }
    else
    {
        return Kp(p, T)*pow(Pstd/(RR*T), nm);
    }
}
```

As introduced in the Mathematical Expression Optimization subsection, an equivalent mathematical expression can reduce the number of exponential evaluations from the number of reversible reactions to the number of species.

In this project, the thermophysical code first computes the dimensionless standard Gibbs free energy for each species; then the reaction-rate code uses vectorized math functions to evaluate the species exponentials; finally, the reaction-rate assembly function individually assembles the Gibbs free-energy change for different elementary reactions.

For an elementary reaction with exactly two reactants and two products, the code for assembling the equilibrium constant and computing the reverse reaction rate is shown below. For brevity, only the complete implementation of the `k+0` reaction block is shown here; the `k+1`, `k+2`, and `k+3` blocks are identical in structure, only with correspondingly incremented index offsets. Here `Kr0` denotes the reverse rate constant of the 0-th reaction, and `invKc0` denotes the reciprocal of the chemical equilibrium constant of the 0-th reaction. `ExpNegGbyRT[sl0a]` is the exponential of the negative dimensionless Gibbs free energy of the `sl0a`-th species: $\exp\left(-\frac{G_{std}}{RuT}\right)$. `invNegGstdByRT` is its reciprocal. `invKcLimiter` is a limiter used to cap `invKc` from growing too large, and its value can be modified.

```
  std::size_t endRR22 = this->RR22Size;
  std::size_t remainRR22 = endRR22 % 4;
  const double invKcLimiter = FastChemistry::invKcLimiter;
  for(std::size_t k=0; k<endRR22-remainRR22; k=k+4)
  {
    // ========== Reaction block k+0 (a) ==========
    const unsigned int i0 = this->RR22AllIndex[k*5+0];
    const unsigned int sl0a = this->RR22AllIndex[k*5+1];
    const unsigned int sl1a = this->RR22AllIndex[k*5+2];
    const unsigned int sr0a = this->RR22AllIndex[k*5+3];
    const unsigned int sr1a = this->RR22AllIndex[k*5+4];
    double Kr0 = 0;
    const double Kf0 = Kf_[i0];
    // assemble the reciprocal of the chemical equilibrium constant
    double invKc0 = (ExpNegGbyRT[sl0a]*ExpNegGbyRT[sl1a])*
(invNegGstdByRT[sr0a]*invNegGstdByRT[sr1a]);
    invKc0 = std::min(invKc0, invKcLimiter);
    // compute the reverse rate constant
    Kr0 = Kf0*invKc0;
    const double CF0 = c[sl0a]*c[sl1a];
    const double CR0 = c[sr0a]*c[sr1a];
```

```
    const double q0 = (Kf0*CF0) - (Kr0*CR0);
    // write the reaction rates into the species production-rate array
    const double omega_sl0a_increment = dNdtByV[sl0a] - q0;
    const double omega_sl1a_increment = dNdtByV[sl1a] - q0;
    const double omega_sr0a_increment = dNdtByV[sr0a] + q0;
    const double omega_sr1a_increment = dNdtByV[sr1a] + q0;
    dNdtByV[sl0a] = omega_sl0a_increment;
    dNdtByV[sl1a] = omega_sl1a_increment;
    dNdtByV[sr0a] = omega_sr0a_increment;
    dNdtByV[sr1a] = omega_sr1a_increment;

    // ========== Reaction block k+1 (b) ==========
```

2.3.4. Assembling the Partial-Derivative Matrix of Reaction Rates w.r.t. Molar Concentrations and Temperature

Assembling the partial-derivative matrix of reaction rates w.r.t. molar concentrations and temperature mainly includes the partial derivative of the rate constant w.r.t. temperature, the partial derivative w.r.t. concentration (for third-body and pressure-dependent reactions), and the partial derivative of the concentration product w.r.t. concentration. The latter is noteworthy. OpenFOAM-10's implementation is as follows; for convenience, only the partial derivative of the reactant concentration product w.r.t. reactant concentration is shown. Here `lhs()` returns the reactant indices and reaction orders. `sj` is the index of the j-th species, and `reduced` is a `bool` variable indicating whether the current reaction mechanism has been reduced online, causing the species list to change. The variable `dCfdcj` is the partial derivative of the reactant concentration product w.r.t. the j-th species; it then enters a loop to compute this partial derivative, where `el` is the reaction order, `pow` is the power operation, and `max` avoids negative concentrations. The code `specieExponent(label(1))` yields an integer or floating-point value of 1. After `dCfdcj` is computed, two further loops write the result into the Jacobian matrix, where `sl` is the stoichiometric coefficient and `kf` is the forward rate constant. For the concentration partial derivative of the product concentration product, and for the rate-constant partial derivatives w.r.t. concentration and temperature, the code is omitted; refer to the `ddNdtByVdcTp` function in Reaction.C.

```
    // Jacobian contributions from the derivative of the concentration products
    // w.r.t. concentration
    {
        forAll(lhs(), j)
        {
            const label sj = reduced ? c2s[lhs()[j].index] : lhs()[j].index;

            scalar dCfdcj = 1;
            forAll(lhs(), i)
            {
                const label si = lhs()[i].index;
                const specieExponent& el = lhs()[i].exponent;
                if (i == j)
                {
                    dCfdcj *=
                        c[si] >= small || el >= 1
```

```
                        ? el*pow(max(c[si], 0), el - specieExponent(label(1)))
                        : 0;
                }
                else
                {
                    dCfdcj *=
                        c[si] >= small || el >= 1
                      ? pow(max(c[si], 0), el)
                      : 0;
                }
            }

            forAll(lhs(), i)
            {
                const label si = reduced ? c2s[lhs()[i].index] : lhs()[i].index;
                const scalar sl = lhs()[i].stoichCoeff;
                ddNdtByVdcTp(Nsi0 + si, Nsi0 + sj) -= sl*kf*dCfdcj;
            }
            forAll(rhs(), i)
            {
                const label si = reduced ? c2s[rhs()[i].index] : rhs()[i].index;
                const scalar sr = rhs()[i].stoichCoeff;
                ddNdtByVdcTp(Nsi0 + si, Nsi0 + sj) += sr*kf*dCfdcj;
            }
        }
    // ... continue processing
```

The problem with the code above is that it has too many nested loops and conditional branches, while elementary reactions usually have no more than 3 reactants and products, and the instructions executed within the loops are too few, making the loop-control and branch overhead disproportionately large. In fact, only global reactions need such a complex expression; elementary reactions can be greatly simplified. For example, for an elementary reaction with two reactants and two products, the expression above can be rewritten as follows:

```
   for(std::size_t k=0; k<endRRDup22-remainRRDup22; k=k+4)
   {
      // ========== Reaction block k+0 (a) ==========
      // extract rate constants, reactant and product indices
      const unsigned int i0 = this->RR22DupAllIndex[k*5+0];
      const unsigned int sl0a = this->RR22DupAllIndex[k*5+1];
      const unsigned int sl1a = this->RR22DupAllIndex[k*5+2];
      const unsigned int sr0a = this->RR22DupAllIndex[k*5+3];
      const unsigned int sr1a = this->RR22DupAllIndex[k*5+4];
      // assemble the partial derivative of the forward reaction rate w.r.t. reactant molar
concentrations, and write into the Jacobian matrix
      const double Kf0 = this->Kf_[i0];
      const double dCfdC0a = C[sl1a];
      const double dCfdC1a = C[sl0a];
      ddNdtByVdcTp[sl0a*(this->alignN)+ sl0a] -= Kf0*dCfdC0a;
      ddNdtByVdcTp[sl1a*(this->alignN)+ sl0a] -= Kf0*dCfdC0a;
      ddNdtByVdcTp[sl0a*(this->alignN)+ sl1a] -= Kf0*dCfdC1a;
```

```
      ddNdtByVdcTp[sl1a*(this->alignN)+ sl1a] -= Kf0*dCfdC1a;
      ddNdtByVdcTp[sr0a*(this->alignN)+ sl0a] += Kf0*dCfdC0a;
      ddNdtByVdcTp[sr1a*(this->alignN)+ sl0a] += Kf0*dCfdC0a;
      ddNdtByVdcTp[sr0a*(this->alignN)+ sl1a] += Kf0*dCfdC1a;
      ddNdtByVdcTp[sr1a*(this->alignN)+ sl1a] += Kf0*dCfdC1a;
      // assemble the partial derivative of the reverse reaction rate w.r.t. product molar
concentrations, and write into the Jacobian matrix
      const double invKp0 = (invNegGstdByRT[sr0a]*invNegGstdByRT[sr1a])*
(ExpNegGbyRT[sl0a]*ExpNegGbyRT[sl1a]);
      double invKc0 = std::min(invKp0,FastChemistry::invKcLimiter);
      const double Kr0 = Kf0*invKc0;
      const double dCrdC0a = C[sr1a];
      const double dCrdC1a = C[sr0a];
      ddNdtByVdcTp[sl0a*(this->alignN)+ sr0a] -= (-Kr0*dCrdC0a);
      ddNdtByVdcTp[sl1a*(this->alignN)+ sr0a] -= (-Kr0*dCrdC0a);
      ddNdtByVdcTp[sl0a*(this->alignN)+ sr1a] -= (-Kr0*dCrdC1a);
      ddNdtByVdcTp[sl1a*(this->alignN)+ sr1a] -= (-Kr0*dCrdC1a);
      ddNdtByVdcTp[sr0a*(this->alignN)+ sr0a] += (-Kr0*dCrdC0a);
      ddNdtByVdcTp[sr1a*(this->alignN)+ sr0a] += (-Kr0*dCrdC0a);
      ddNdtByVdcTp[sr0a*(this->alignN)+ sr1a] += (-Kr0*dCrdC1a);
      ddNdtByVdcTp[sr1a*(this->alignN)+ sr1a] += (-Kr0*dCrdC1a);
      // assemble the partial derivative of the reaction rate w.r.t. temperature, and write
into the Jacobian matrix
      const double dKfdT0 = this->dKfdT_[i0];
      const double sumVdBdT0 = (dBdT[sr0a] + dBdT[sr1a]) - (dBdT[sl0a] + dBdT[sl1a]);
      const double dKcdTByKc0 = sumVdBdT0;
      const double dKrdT0 = (dKfdT0*invKc0 - (invKc0 < FastChemistry::invKcLimiter ?
Kr0*dKcdTByKc0 : 0));
      const double dqdT0 = (dKfdT0*CFa) - (dKrdT0*CRa);
      ddNdtByVdcTp[sl0a*(this->alignN)+(this->nSpecies)] -= dqdT0;
      ddNdtByVdcTp[sl1a*(this->alignN)+(this->nSpecies)] -= dqdT0;
      ddNdtByVdcTp[sr0a*(this->alignN)+(this->nSpecies)] += dqdT0;
      ddNdtByVdcTp[sr1a*(this->alignN)+(this->nSpecies)] += dqdT0;
      // assemble the reaction rates and write into the species production-rate array
      const double CFa = C[sl0a]*C[sl1a];
      const double CRa = C[sr0a]*C[sr1a];
      const double q0 = (Kf0*CFa) - (Kr0*CRa);
      dNdtByV[sl0a] = dNdtByV[sl0a] - q0;
      dNdtByV[sl1a] = dNdtByV[sl1a] - q0;
      dNdtByV[sr0a] = dNdtByV[sr0a] + q0;
      dNdtByV[sr1a] = dNdtByV[sr1a] + q0;
      //...  assemble the partial derivative of the rate constant w.r.t. molar concentration
(for third-body and pressure-dependent reactions)
```

Compared with the complex nested loops of the general version, the unrolled code has the following advantages:

Eliminating loop-carried data dependency: in the general code, the variable `dCfdcj` is repeatedly multiplied within the loop. The result of the previous multiplication must be written back to a register before the next multiplication can read it, forming a long serial latency-dependency chain. In the unrolled code, however, `dCfdC0a` and `dCfdC1a` come from two completely independent load instructions (`C[sl1a]` and `C[sl0a]`),

with no data dependency between them. The CPU's out-of-order execution engine can assign them to two different floating-point multiplication units in parallel, hiding the load latency.

Eliminating control hazards: in the general code, branches such as `if (i == j)` and `if (c[si] >= small)` may change frequently across loop iterations, making it hard for the branch predictor to hit stably. Once a misprediction occurs, the CPU must flush the prefetched ten-odd pipeline stages (Pipeline Flush), at a penalty of up to a dozen clock cycles. The unrolled code is a simple code block with no branch instructions. The CPU front end can fetch and decode continuously without obstruction, keeping the pipeline fully utilized; the back-end execution units never idle due to branch misprediction.

Simultaneous instruction issue: in the unrolled code, the three groups—forward-rate partial derivatives, reverse-rate partial derivatives, and temperature partial derivatives—have very little mutual dependency, both within and across groups. A modern CPU can in principle execute these independent instructions simultaneously, whereas the original code consists of almost independent loops. For loops, since a conventional CPU normally has only one program counter per core, it cannot fetch two independent loops in parallel—they are two independent control flows; while hyper-threading can realize independent control flows, the code above is not multi-threaded.

Eliminating exponentiation: in the general code, the `pow` used to compute concentration powers still contains a loop structure even with the fast-power algorithm for integer exponents (using shift operations); each iteration involves shifting, comparison, and conditional branching. For the common exponents 1, 2, 3 in elementary reactions, although the loop count is small, there is still branch and loop-control overhead that interrupts the continuous issue of the pipeline; moreover, the function must maintain a temporary accumulator variable, introducing unnecessary data dependency, while for exponent 1 the operation could be omitted entirely.

2.3.5. Thermophysical Property Computation

Computing reaction rates requires two thermophysical quantities—enthalpy and specific heat. Computing the Jacobian matrix also requires the partial derivative of specific heat w.r.t. temperature, the Gibbs free energy, and the partial derivative of Gibbs free energy w.r.t. temperature. A fragment of OpenFOAM-10 code is shown below, where Cp is the species constant-pressure specific heat, CpM is that of the mixture, and dCpMdT is the partial derivative of the mixture constant-pressure specific heat w.r.t. temperature.

```
// Evaluate the mixture Cp and its derivative
scalarField& Cp = YTpYTpWork_[3];
scalar CpM = 0, dCpMdT = 0;
for (label i=0; i<Y_.size(); i++)
{
    Cp[i] = specieThermos_[i].Cp(p, T);
    CpM += Y_[i]*Cp[i];
    dCpMdT += Y_[i]*specieThermos_[i].dCpdT(p, T);
}
```

A fragment using the janaf polynomial to compute the constant-pressure specific heat is shown below:

```
template<class EquationOfState>
inline Foam::scalar Foam::janafThermo<EquationOfState>::Cp
(
    const scalar p,
    const scalar T
) const
{
    const coeffArray& a = coeffs(T);
    return
        ((((a[4]*T + a[3])*T + a[2])*T + a[1])*T + a[0])
      + EquationOfState::Cp(p, T);
}
```

The call to the function `coeffs(T)` can be further optimized. This function determines whether the current temperature lies in the low-temperature or high-temperature range and then returns the corresponding polynomial coefficients. We identified the following improvable points:

1. Every time a thermophysical quantity is computed, an interval check is invoked; however, after the specific heat is computed, the enthalpy computation no longer needs to re-check the interval.
2. If we know the current input temperature is smaller than the minimum `Tcommon` across all species, then we need not check each species' `Tcommon` one by one.
3. The janaf expression can be vectorized to compute the thermophysical quantities of multiple species at once.

The code for obtaining the janaf polynomial coefficients is shown below. Each species retains its own independent `Tcommon` (the boundary temperature between the high- and low-temperature ranges). `regime` is used only to judge the overall temperature distribution relative to all species' `Tcommon`: if the temperature is above all species' `Tcommon` or below all species' `Tcommon`, the corresponding coefficients can be loaded directly; if it lies in the mixed range, element-wise selection via `mask` is required. This classification strategy reduces the invocation frequency of `mask` operations.

```
__m256d A0,A1,A2,A3,A4,A5,A6;

if (regime == 2)
{
const __m256d mask = _mm256_cmp_pd(vT,load256d(&this->TcommonSoA[i]),_CMP_LT_OQ);
A0 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[0][i]), load256d(&this->LCoeffsSoA[0][i]),
mask);
A1 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[1][i]), load256d(&this->LCoeffsSoA[1][i]),
mask);
A2 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[2][i]), load256d(&this->LCoeffsSoA[2][i]),
mask);
A3 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[3][i]), load256d(&this->LCoeffsSoA[3][i]),
mask);
A4 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[4][i]), load256d(&this->LCoeffsSoA[4][i]),
mask);
A5 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[5][i]), load256d(&this->LCoeffsSoA[5][i]),
mask);
A6 = _mm256_blendv_pd(load256d(&this->HCoeffsSoA[6][i]), load256d(&this->LCoeffsSoA[6][i]),
mask);
```

```
}
else if(regime == 1)//T>max(Tcommon)
{
A0 = load256d(&this->HCoeffsSoA[0][i]);
A1 = load256d(&this->HCoeffsSoA[1][i]);
A2 = load256d(&this->HCoeffsSoA[2][i]);
A3 = load256d(&this->HCoeffsSoA[3][i]);
A4 = load256d(&this->HCoeffsSoA[4][i]);
A5 = load256d(&this->HCoeffsSoA[5][i]);
A6 = load256d(&this->HCoeffsSoA[6][i]);
}
else//T<min(Tcommon)
{
A0 = load256d(&this->LCoeffsSoA[0][i]);
A1 = load256d(&this->LCoeffsSoA[1][i]);
A2 = load256d(&this->LCoeffsSoA[2][i]);
A3 = load256d(&this->LCoeffsSoA[3][i]);
A4 = load256d(&this->LCoeffsSoA[4][i]);
A5 = load256d(&this->LCoeffsSoA[5][i]);
A6 = load256d(&this->LCoeffsSoA[6][i]);
}
```

The vectorized code is shown below; it illustrates the computation of the partial derivative of constant-pressure specific heat w.r.t. temperature, and the vectorized evaluation of constant-pressure specific heat and specific absolute enthalpy.

```
        __m256d dCpdTv = _mm256_fmadd_pd(vT,4*A4,3*A3);
        dCpdTv = _mm256_fmadd_pd(vT,dCpdTv,2*A2);
        dCpdTv = _mm256_fmadd_pd(vT,dCpdTv,A1);
        dCpdTv = _mm256_mul_pd(Ruv,dCpdTv);
        store256d(&dCpdT[i],dCpdTv);

        __m256d Cpv = _mm256_fmadd_pd(A4 ,vT,A3);
        Cpv = _mm256_fmadd_pd(Cpv,vT,A2);
        Cpv = _mm256_fmadd_pd(Cpv,vT,A1);
        Cpv = _mm256_fmadd_pd(Cpv,vT,A0);
        Cpv = _mm256_mul_pd(Ruv,Cpv);
        store256d(&Cp[i],Cpv);

        __m256d Hav = _mm256_fmadd_pd(A4 ,vT*0.2,A3*0.25);
        Hav = _mm256_fmadd_pd(Hav,vT,A2*(1.0/3.0));
        Hav = _mm256_fmadd_pd(Hav,vT,A1*0.5);
        Hav = _mm256_fmadd_pd(Hav,vT,A0);
        Hav = _mm256_fmadd_pd(Hav,vT,A5);
        Hav = _mm256_mul_pd(Ruv,Hav);
        store256d(&Ha[i],Hav);
```

2.3.6. Assembling the Jacobian Matrix

For a chemical-reaction system using mass fraction as the state variable, the Jacobian matrix computation has complexity ($O(Ns^3)$). Therefore, as the number of species (Ns) in the mechanism increases, the Jacobian workload grows rapidly, and targeted performance optimization is especially needed for large mechanisms. The formula is:

$$\begin{cases} \dfrac{\partial}{\partial Y_j}\left(\dfrac{dY_i}{dt}\right) = \dfrac{W_i}{\rho}\displaystyle\sum_{k=1}^{N_s}\dfrac{\partial \dot{\omega}_i}{\partial C_k}\dfrac{\partial C_k}{\partial Y_j} - \dfrac{dY_i}{dt}\dfrac{1}{\rho}\dfrac{\partial \rho}{\partial Y_j} \\ \dfrac{\partial C_i}{\partial Y_j} = \dfrac{\partial \rho}{\partial Y_j}\dfrac{Y_i}{W_i} + \dfrac{\rho}{W_i}\delta_{ij} \end{cases} \tag{90}$$

The above computation can essentially be expressed as matrix multiplication and element-wise operations. Therefore, the optimization methods commonly used in high-performance matrix multiplication also apply here, including blocking to improve cache locality, loop unrolling and instruction-level parallelism to raise execution-unit utilization, and explicit SIMD instructions for vectorization.

In practical 3-D reacting-flow computation, engineers usually do not use mechanisms with **hundreds** of species, so a single matrix is relatively small. In this case, dedicated matrix packing to improve cache access yields limited benefit, while extra data rearrangement introduces overhead. Therefore, this project does not adopt the matrix-packing strategy of general matrix multiplication; instead, it directly performs blocked computation for the fixed data layout. Meanwhile, the unrolling factor of the three nested loops must be chosen sensibly to maximize register and execution-unit utilization without causing register spill due to excessive register pressure.

In addition, **operator fusion** can be applied. OpenFOAM's original implementation first computes the full

$$\frac{\partial C_i}{\partial Y_j} \tag{91}$$

matrix and then uses it to compute the mass-fraction Jacobian matrix

$$\frac{\partial}{\partial Y_j}\left(\frac{dY_i}{dt}\right) \tag{92}$$

This implementation must write the intermediate matrix entirely to memory and then read it back with strided access in later computation (possibly a cache miss). Since each column of $\frac{\partial C_i}{\partial Y_j}$ can be used independently to compute the corresponding Jacobian column, this project fuses the two operations: after computing one column of $\frac{\partial C_i}{\partial Y_j}$, it is immediately used for the corresponding column of the mass-fraction Jacobian, instead of writing the full intermediate matrix to memory. This eliminates the complete write-back and reload of the intermediate matrix, reduces memory access, and improves the reuse rate of data in cache or registers.

In summary, this optimization does not target a single instruction but combines blocking, loop unrolling, instruction-level parallelism, SIMD vectorization, and operator fusion to optimize the entire mass-fraction Jacobian computation. Because the final implementation involves many explicit SIMD instructions, loop unrolling, and data-layout optimizations for specific matrix sizes—differing greatly from a general implementation and being less readable—the complete optimized implementation is not shown here.

For the mole-number-based Jacobian computation, the time complexity drops from ($O(N_s^3)$) of the mass-fraction Jacobian to ($O(N_s^2)$); thus, as the species count grows, the workload increases much more slowly and it is no longer the main performance bottleneck of the whole chemical computation. Based on this, this project does not perform complex low-level optimization for this part, but mainly uses blocking, loop unrolling, and SIMD vectorization to raise instruction-level parallelism and execution-unit utilization. In addition, the four Jacobian sub-matrices—species w.r.t. species, species w.r.t. temperature, temperature w.r.t. species, and temperature w.r.t. temperature—are fused and updated within the same outer loop, reducing loop-control overhead and improving data locality and reuse. Since this part has low complexity and no longer forms a main bottleneck after the above optimization, its detailed implementation code is omitted.

2.3.7. LU Decomposition

OpenFOAM-10's native version implements LU decomposition with a simple scalar loop. To accelerate it, a block algorithm can be used. Although the block algorithm does not reduce the theoretical time complexity of LU decomposition, it fits high-performance computing code very well and consists mainly of four steps; beforehand, a stability treatment is also needed to reduce round-off error. For simplicity, this document **ignores the pivoting process**—the solver's actual blocked computation does perform pivoting.

Step zero: stability treatment

To improve numerical stability, we introduce **row-equilibration stability treatment** in the LU decomposition: for each row, divide all elements of that row by the maximum absolute value among them. This effectively reduces the disparity of floating-point magnitudes in the matrix and suppresses round-off error propagation caused by magnitude differences, thereby ensuring numerical robustness of the solution. It is equivalent to the following operation:

$$\mathbf{R}\mathbf{A}x = \mathbf{R}b \tag{93}$$

This makes the values in each row of the matrix close to one another, avoiding round-off error.

Step one: matrix blocking

Partition the global coefficient matrix $\mathbf{A}$ into four sub-blocks by degrees of freedom or mesh regions:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix} \tag{94}$$

where $\mathbf{A}_{11}$ is a square matrix (its size usually matches the cache capacity), $\mathbf{A}_{12}$ and $\mathbf{A}_{21}$ are sub-matrices, and $\mathbf{A}_{22}$ is the remaining square matrix.

Step two: factor the diagonal block $\mathbf{A}_{11}$

Perform scalar LU decomposition on $\mathbf{A}_{11}$ to obtain the lower-triangular factor $\mathbf{L}_{11}$ and the upper-triangular factor $\mathbf{U}_{11}$, i.e.:

$$\mathbf{A}_{11} = \mathbf{L}_{11}\mathbf{U}_{11} \tag{95}$$

Step three: solve for $\mathbf{U}_{12}$ and $\mathbf{L}_{21}$

Using the already-factored $\mathbf{L}_{11}$ and $\mathbf{U}_{11}$, solve the two triangular systems respectively:

- Solve for $\mathbf{U}_{12}$: compute $\mathbf{A}_{12}$ into $\mathbf{U}_{12}$ to obtain the block upper-triangular part;

$$\mathbf{L}_{11}\mathbf{U}_{12} = \mathbf{A}_{12} \tag{96}$$

- Solve for $\mathbf{L}_{21}$: forward substitution yields the block lower-triangular part.

$$\mathbf{L}_{21}\mathbf{U}_{11} = \mathbf{A}_{21} \tag{97}$$

Step four: compute $\mathbf{L}_{22}\mathbf{U}_{22}$

$$\mathbf{L}_{22}\mathbf{U}_{22} = \mathbf{A}_{22} - \mathbf{L}_{21}\mathbf{U}_{12} \tag{98}$$

Step four involves matrix multiplication, which can be accelerated with a well-structured high-performance matrix multiplication.

To illustrate the effect of round-off error, consider the DRM19 counterflow-flame case in this project. In this mechanism file, the last species is AR, and the state-variable ordering is: the second-to-last is **temperature**, the last is **the mass fraction (or mole number) of AR**. In the DRM19 mechanism, AR is an inert species that participates in no chemical reaction; its net production rate is theoretically always zero. In its corresponding Jacobian row, only the diagonal entry is non-zero, and the off-diagonal entries differ greatly in magnitude from other reactive species. An LU decomposition without row equilibration will propagate round-off errors from other rows/columns into the AR row during back-substitution, producing spurious production rates and breaking AR conservation.

The table below shows the effect of the LU-decomposition stability treatment on this case. The comparison procedure is as follows: first, advance 5000 flow time steps using the **row-equilibrated** LU decomposition, writing the result to a file as a common initial state. Then restart from that state and advance 2 time steps each using the **row-equilibrated** and **non-row-equilibrated** LU decompositions, recording the **initial residual of the species and energy conservation equations before the iterative solve** at step 2 (i.e., global step 5002). This ensures the two comparisons start from identical initial physical states, with the difference coming only from the LU-decomposition solver itself. The results show that the row-equilibrated LU decomposition reduces the linear-system residual by 1–2 orders of magnitude, clearly indicating that the numerical "spikes" in the time-averaged reaction source terms have been effectively removed and that convergence is significantly improved.

Note on "initial residual": the "initial residual" reported in the tables of this section refers to the residual norm at the start of the iterative solve for the **flow-transport step** (i.e., the coupled system of species transport, momentum, and continuity equations). Specifically, this residual reflects how well the governing equations are satisfied on the initial guessed solution after the current chemical source terms are substituted into the transport equations. Since the accuracy of chemical source-term computation directly affects the convergence behavior of the whole flow system, comparing the flow-transport-step initial residuals under row-equilibrated and non-row-equilibrated LU decompositions indirectly assesses the impact of row-equilibration preprocessing on the accuracy of chemical reaction source-term computation.

Variable	Initial Residual at Step 5000	Initial Residual at Step 5002 (with Stabilization)	Initial Residual at Step 5002 (without Stabilization)
H2	8.07773669639e-07	4.30124103083e-06	0.00023164216566
H	7.80969092974e-07	2.91239503475e-06	0.00154060474534
O	1.98094585301e-06	8.41770946852e-06	0.00113068470141
O2	8.37850193577e-07	5.00290797983e-06	0.00011452575272
OH	1.56479251388e-06	6.18866739504e-06	0.00123186192617
H2O	1.4500857788e-06	1.09362833098e-05	0.000255630009979
HO2	2.26632341066e-05	0.000427909268764	0.00113436504584

Variable	Initial Residual at Step 5000	Initial Residual at Step 5002 (with Stabilization)	Initial Residual at Step 5002 (without Stabilization)
CH2	6.04221982035e-07	1.94448687304e-06	0.00107339664717
CH2(S)	1.06982869147e-06	1.52124347686e-06	0.00106812524545
CH3	7.32043336772e-07	1.19822430117e-05	0.000819589355864
CH4	8.75685315901e-07	2.64002984676e-06	5.3969668832e-05
CO	8.10151362475e-07	4.20672358864e-06	0.000252073526698
CO2	2.03249858563e-06	1.39036477485e-05	0.000291872736112
HCO	7.59772199864e-07	1.39072706728e-05	0.000906916870092
CH2O	1.16154769628e-06	8.41567327366e-06	0.00043624112379
CH3O	9.60075123264e-07	8.83747493013e-05	0.000682376878019
C2H4	6.44086665082e-07	4.47106504127e-06	0.000240535207287
C2H5	1.12158266659e-06	7.85939074991e-05	0.000822157536816
C2H6	4.91956325537e-07	1.4964665166e-05	0.000223344525865
AR	0	0	0.259775542871
h	1.66159049361e-06	1.20248052255e-05	0.000256259880695

OpenFOAM-10's standard LU decomposition adopts a **global row-pivoting** strategy, which is theoretically more numerically stable. The blocked LU decomposition used in this solver performs pivoting inside each block; although slightly inferior to global pivoting in theoretical stability, it greatly improves cache locality and parallelism, and the row-equilibration preprocessing already largely mitigates the numerical-scale problem. The data of Table 1 show that, in the current case, after row equilibration the solver's residual has dropped to an acceptable range.

2.3.8. LU Solve

After obtaining a linear system of the form (LUx=b), it must be solved via forward and back substitution for (x). For the off-diagonal blocks, their regular data-access pattern allows the matrix-vector multiply-add operations to be SIMD-vectorized, improving efficiency.

For the triangular solve of the diagonal block, strong data dependencies exist among adjacent unknowns during forward and back substitution. To weaken this serial dependency, the technique of **delayed horizontal reduction** can be used: instead of immediately reducing each intermediate result to a scalar, it is represented as a SIMD vector containing multiple partial sums, so that subsequent computations can directly use these vectors for multiply-add operations, and a horizontal sum is performed only at the end.

Taking the forward substitution of a (4×4) lower-triangular matrix as an example, its scalar form is:

```
y0 = b0;
y1 = b1 - L10*y0;
y2 = b2 - L20*y0 - L21*y1;
y3 = b3 - L30*y0 - L31*y1 - L32*y2;
```

After applying delayed horizontal reduction, the intermediate results can be represented as SIMD vectors. The initialization is:

```
y0  = [b0, 0,  0,  0];
y1  = [0,  b1, 0,  0];
y2  = [0,  0,  b2, 0];
y3  = [0,  0,  0,  b3];

L10v = [L10, L10, L10, L10];
L20v = [L20, L20, L20, L20];
L21v = [L21, L21, L21, L21];
L30v = [L30, L30, L30, L30];
L31v = [L31, L31, L31, L31];
L32v = [L32, L32, L32, L32];
```

Then only element-wise multiply-add operations are performed, with no horizontal reduction yet:

```
y1 = y1 - L10v*y0;
y2 = y2 - L20v*y0 - L21v*y1;
y3 = y3 - L30v*y0 - L31v*y1 - L32v*y2;
```

Finally, a horizontal sum is performed on each vector:

```
y0 = hsum(y0);
y1 = hsum(y1);
y2 = hsum(y2);
y3 = hsum(y3);
```

In this way, the computation that originally required an immediate horizontal sum at each step is deferred until all triangular recurrence calculations are finished, reducing the serial dependency introduced by horizontal reduction.

In the actual AVX2 implementation, the required SIMD vectors can be constructed via consecutive loads combined with `shuffle` instructions, and a (4×4) horizontal sum designed for multiple vectors reduces the intermediate results to the final four scalars at once, further improving the efficiency of the forward-substitution process.

2.3.9. Memory Reuse

The purpose of memory reuse is to reduce the cache-miss rate by decreasing the number of resident matrices and optimizing the data-access pattern. Specifically:

1. Matrix reuse keeps the same data "hot" in cache across multiple computations, avoiding repeated loads.
2. In-place updates eliminate extra memory allocation and data copies.
3. Data access is concentrated in fewer memory regions, improving the spatial locality of cache lines.

Taking the Rodas34 integration algorithm as an example, the chemical-reaction solve in OpenFOAM uses at least four matrices, each with its own memory space, as follows. Here `dcdY` is the partial-derivative matrix of molar concentration w.r.t. species mass fraction, `ddNdtByVdcTp` is the partial-derivative matrix of molar-concentration rate w.r.t. molar concentration and temperature, `dfdy_` is the Jacobian matrix of species mass-fraction and temperature rates, and `a_` is $\mathbf{I}/(\gamma\Delta t) - \mathbf{J}^N$ in the linearly implicit method.

```
    // inside the Jacobian function in chemistryModel.C
```

```
    scalarSquareMatrix& dcdY = YTpYTpWork_[0];
    scalarSquareMatrix& ddNdtByVdcTp = YTpYTpWork_[1];

    // inside the solve function in rodas34.C
    odes_.Jacobian(x0, y0, li, dfdx_, dfdy_);

    for (label i=0; i<n_; i++)
    {
        for (label j=0; j<n_; j++)
        {
            a_(i, j) = -dfdy_(i, j);
        }

        a_(i, i) += 1.0/(gamma*dx);
    }
```

Because this solver uses operator fusion, it avoids the `dcdY` matrix and merges the operations on `a_` and `dfdy_` into a single matrix, reducing the matrix count from 4 to 2: `ddNdtByVdcTp` and `a_`. Here `a_` first serves as the Jacobian matrix and then, through an identity-matrix operation, becomes the target matrix of the LU decomposition.

If mole number is chosen as the state variable, the `ddNdtByVdcTp` matrix can be converted in place directly into the Jacobian matrix and then reused as the `a_` matrix. Under this scheme, the entire time-advance process needs to maintain only one matrix (namely `a_`), reducing memory usage from 4 matrices to 1 and significantly lowering cache pressure (for the Seulex method, two matrices must be retained simultaneously—one Jacobian matrix and one `a_` matrix).

3. Tests

3.1. Setup

- All PLOG reactions are commented out and converted into conventional Arrhenius reactions. Note that this solver supports PLOG reactions, but the file must be modified manually.
- The format of all reaction mechanism files was manually modified to ensure successful conversion into Foam mechanism files by the ChemkinToFoam utility.
- In the counterflow flame cases, the lower bound of the JANAF temperature range for all mechanisms was modified from 300 K or 298.15 K to 280 K.
- In the CH3OH_116s_962r case, the upper bound of the mechanism temperature range was raised to 2500 K; some species originally had an upper bound of only 1500 K or 2000 K.
- In the DME_26s_71r case, `T***` and `T*` were changed from 0 to 1e-30 to avoid floating-point overflow.
- In the DME_39s_175r case, `T***` and `T*` were changed from 0 to 1e-30 to avoid floating-point overflow.
- In the NC7H16_32s_104r case, the N2 species was moved to the first position in the species list. Using NC7H16 as the first species would cause an error when computing temperature from energy; this error originates from the way OpenFOAM-10's thermophysical library computes the JANAF polynomial for the mixture.
- In the NH3CH4_125s_1099r case, the upper bound of the temperature range for some species was raised from 2000 K to 2500 K.

3.2. Code Validation Tests

This chapter uses zero-dimensional ignition cases for the validation tests, executed with chemFoam. The number of ODE time steps required to complete each case is counted and compared against the official OpenFOAM-10 solver. The OpenFOAM-10 solver uses an exact Jacobian matrix, exact pressure-dependent reaction calculations, and a corrected reaction source term that removes the effect of volume change. The results are shown in the table below; the iteration counts of this solver differ from those of the official OF10 solver only by a small margin. The absolute convergence tolerance is 10^{-12} and the relative convergence tolerance is 10^{-6}.

For more detailed tests, please refer to the paper on this solver:

Chi, Z. (2025). A high-performance chemistry solver for reactive flow simulations in OpenFOAM. *Physics of Fluids*, *37*(12), 125154. https://doi.org/10.1063/5.0250891

	A12R6	**A12R6**	**A12R6**
	OF10	This solver (mass fraction)	This solver (mole number)
C2H4_59s_271r	1435	1434	1434
C2H4_25s_70r	1165	1165	1165
C2H5OH_57s_383r	1535	1510	1510
C2H5OH_39s_370r	1694	1691	1691
C3H8_70s_463r	3357	3357	3357
CH3OH_116s_962r	1305	1303	1303
CH4_21s_84r	2007	2007	2007
CH4_28s_116r	1784	1784	1784
CH4_38s_291r	2212	2212	2212
CH4_53s_325r	2031	2031	2031
CH4_68s_631r	2211	2211	2211
CH4_59s_271r	2202	2196	2197
DME_26s_71r	1786	1777	1777
DME_79s_351r	2545	2525	2525
DME_39s_175r	2003	2002	2002
H2_13s_27r	458	458	458
H2_10s_21r	432	426	426
H2_15s_71r	512	512	512
IC8H18_140s_643r	2678	2674	2674
Kerosene_28s_92r	6103	6089	6089

	A12R6	A12R6	A12R6
Kerosene_32s_77r	4208	4208	4208
NC12H26_31s_193r	2275	2275	2275
NC12H26_54s_269r	2692	2682	2682
NC7H16_46s_115r	1501	1501	1501
NC7H16_68s_283r	1780	1770	1770
NC7H16_32s_104r	1969	1964	1964
NH3CH4_129s_1231r	2139	2140	2140
NH3CH4_59s_356r	2094	2091	2091
NH3CH4_42s_130r	2273	2273	2273
NH3CH4_125s_1099r	2216	2210	2210
NH3_38s_232r	2812	2813	2813
NH3_32s_213r	1637	1633	1633
NH3_31s_200r	1232	1234	1234
NH3_38s_263r	1066	1060	1060

3.3. Computational Performance Tests

This chapter uses counterflow flames for the performance tests. The processor is an AMD Ryzen 7 9700X 8-Core Processor, the compiler is g++ 15.2.0, and the compilation flags are `-O3 -mavx2  -mfma -lmvec -mno-avx256-split-unaligned-load`.

The code that measures execution time is located in the laminar file of CombustionModel, as shown below:

```
auto TimeStart = std::chrono::high_resolution_clock::now();
        // compute time-averaged reaction rate
        chemistryPtr_->solve(min(deltaT, maxIntegrationTime_));
auto duration = (std::chrono::duration_cast<std::chrono::microseconds>
            (std::chrono::high_resolution_clock::now()-TimeStart));
Info<<duration.count()<<endl;
```

The absolute and relative tolerances are 10^{-8} and 10^{-2}, the algorithm is Rodas34, and this project's solver uses mole number as the state variable. The OpenFOAM-10 solver uses an exact Jacobian matrix, exact pressure-dependent reaction calculations, and a corrected reaction source term that removes the effect of volume change.

OpenFOAM-10 encountered an error when computing the C2H5OH_39s_370r case; after solving the linear system, the energy-to-temperature solve diverged. This is related to the code implementation of the JANAF polynomial for the mixture.

	This solver elapsed time (μs)	OpenFOAM-10 solver elapsed time (μs)	Speedup
C2H4_59s_271r	570566	6469354	11.33
C2H4_25s_70r	163503	1058164	6.47
C2H5OH_57s_383r	589172	7363595	12.50
C2H5OH_39s_370r	Error	Error	None
C3H8_70s_463r	695488	9697255	13.94
CH3OH_116s_962r	1787935	31575099	17.66
CH4_21s_84r	123825	1148080	9.27
CH4_28s_116r	170910	1783326	10.43
CH4_38s_291r	314786	3658579	11.62
CH4_53s_325r	429124	5244003	12.22
CH4_68s_631r	769305	10295521	13.38
CH4_59s_271r	486231	5552510	11.42
DME_26s_71r	155293	1002588	6.46
DME_79s_351r	759854	9756297	12.84
DME_39s_175r	264638	2660317	10.05
H2_13s_27r	80985	443722	5.48
H2_10s_21r	68141	287171	4.21
H2_15s_71r	118673	954998	8.05
IC8H18_140s_643r	2755356	47184705	17.12
Kerosene_28s_92r	179982	1563087	8.68
Kerosene_32s_77r	169977	1145562	6.74
NC12H26_31s_193r	292448	3013356	10.30
NC12H26_54s_269r	525439	6080154	11.57
NC7H16_46s_115r	272103	2427304	8.92
NC7H16_68s_283r	254487	2365837	9.30
NC7H16_32s_104r	156469	1464974	9.36
NH3CH4_129s_1231r	2135989	39748675	18.61
NH3CH4_59s_356r	490029	6133213	12.52
NH3CH4_42s_130r	221320	2471343	11.16
NH3CH4_125s_1099r	1948508	35145765	18.03
NH3_38s_232r	248308	2729161	10.99
NH3_32s_213r	208019	2144796	10.31
NH3_31s_200r	173150	1827250	10.55
NH3_38s_263r	246064	2725745	11.08

4. References

1. OpenFOAM Foundation. chemistryModel: Change of state variable to mass fraction. OpenFOAM-10. Commit 4f0dfc3bdf09ae6cf9caf0b806af6fabf3002186. GitHub. https://github.com/OpenFOAM/OpenFOAM-10/commit/4f0dfc3bdf09ae6cf9caf0b806af6fabf3002186 ↩

2. Lu, T., & Law, C. K. (2009). Toward accommodating realistic fuel chemistry in large-scale computations. *Progress in Energy and Combustion Science*, *35*(2), 192–215. https://doi.org/10.1016/j.pecs.2008.10.002 ↩